



Επεξεργασία Φωνής και Φυσικής Γλώσσας

8ο εξάμηνο, Ακαδημαϊκή περίοδος 2024-2025

2η Εργαστηριακή Άσκηση

Αγγελική Ζέρβα: 03121101

Δημήτρης Καμπανάκης: 03121012

Περιεχόμενα

1 Προεπεξεργασία Δεδομένων	2
1.1 Κωδικοποίηση Επισημειώσεων (Labels)	2
1.2 Λεκτική Ανάλυση (Tokenization)	2
1.3 Κωδικοποίηση Παραδειγμάτων (Λέξεων)	3
2 Μοντέλο	5
2.1 Embedding Layer	5
2.2 Output Layer(s)	6
2.3 Forward Pass	7
3 Διαδικασία Εκπαίδευσης	8
3.1 Φόρτωση Παραδειγμάτων (DataLoaders)	8
3.2 Βελτιστοποίηση	9
3.3 Εκπαίδευση	10
3.4 Αξιολόγηση	12
4 Κατηγοριοποίηση με χρήση LLM	14
5 Ερωτήματα	17
5.1 Ερώτημα 1	17
5.2 Ερώτημα 2	19
5.3 Ερώτημα 3	21
5.4 Ερώτημα 4	24
5.5 Ερώτημα 5	26
5.6 Ερώτημα 6	29
5.7 Ερώτημα 7	32
5.8 Ερώτημα 8	34

Προπαρασκευή

1 Προεπεξεργασία Δεδομένων

1.1 Κωδικοποίηση Επισημειώσεων (Labels)

Στο πρώτο βήμα χρησιμοποιούμε τη συνάρτηση `LabelEncoder()` της βιβλιοθήκης `scikit-learn`, έτσι ώστε να κωδικοποιήσουμε κάθε κλάση (`positive`, `negative`, `neutral`...) μέσω της μετατροπής από `string` σε αριθμό. Σημειώνουμε ότι για το `Semeval2017A` έχουμε τρεις κλάσεις (`positive`, `negative`, `neutral`), ενώ για το `MR` έχουμε δύο κλάσεις (`positive`, `negative`).

Ζητούμενο 1: Παρακάτω παρουσιάζονται τα πρώτα 10 labels από τα δεδομένα εκπαίδευσης και τις αντιστοιχίες τους σε αριθμούς:

```
First 10 labels (coded): [1 1 0 1 1 2 2 1 2 2]
Corresponding labels: ['neutral' 'neutral' 'negative' 'neutral' 'neutral' 'positive' 'positive']
```

Figure 1: Label Encodings for Semeval2017A

```
First 10 labels (coded): [1 1 1 1 1 1 1 1 1 1]
Corresponding labels: ['positive' 'positive' 'positive' 'positive' 'positive' 'positive']
```

Figure 2: Label Encodings for MR

1.2 Λεκτική Ανάλυση (Tokenization)

το βήμα αυτό, κάθε κείμενο που περιέχεται στο σύνολο δεδομένων μετατρέπεται από μία απλή ακολουθία χαρακτήρων σε μία πιο δομημένη και κατανοήσιμη για το μοντέλο ακολουθία από όρους (`tokens` ή `terms`). Οι όροι αυτοί μπορεί να είναι λέξεις, σημεία στίξης, αριθμοί ή άλλες λεκτικές μονάδες με σημασιολογικό ή συντακτικό ρόλο. Για τη διαδικασία του `tokenization` χρησιμοποιείται η συνάρτηση `word_tokenize` από τη βιβλιοθήκη `NLTK` (`Natural Language Toolkit`). Η συγκεκριμένη μέθοδος βασίζεται σε προκαθορισμένους κανόνες και γλωσσικά πρότυπα, ώστε να εντοπίζει σωστά τα όρια των λέξεων σε προτάσεις της Αγγλικής γλώσσας. Έχει την ικανότητα να χειρίζεται βασικές περιπτώσεις στίξης, συντμήσεων και ειδικών χαρακτήρων, παρέχοντας μια ικανοποιητική πρώτη μορφή επεξεργασίας του κειμένου. Παρακάτω φαίνεται ενδεικτικά το σημείο του κώδικα που προσθήσαμε στο αρχείο `dataloading.py`.

```
# EX2
self.data = X
# Tokenize the sentences with nltk tokenizer
self.tokenized_data = [word_tokenize(sentence) for sentence in self.data]
self.labels = y
self.word2idx = word2idx
```

Ζητούμενο 2 : Παρακάτω παρουσιάζονται δέκα ενδεικτικά παραδείγματα της διαδικασίας `tokenization` για το εκάστοτε σύνολο δεδομένων. Για κάθε πρόταση του αρχικού κειμένου, εμφανίζεται η ακολουθία των `tokens` που προκύπτει μετά την εφαρμογή της συνάρτησης που προσθήσαμε.

Example 1 Token indices: ['Won', 'the', 'match', '#', 'getin', '.', 'Plus\u002c', 'tomorrow', 'is', 'a', 'very', 'busy', 'day\u002c', 'with', 'Awareness', 'Day\u0029s', 'and', 'debates', '.', 'Gulp', '.', 'Debates', '...']

Example 2 Token indices: ['Some', 'areas', 'of', 'New', 'England', 'could', 'see', 'the', 'first', 'flakes', 'of', 'the', 'season', 'Tuesday', '.']

Example 3 Token indices: ['@', 'francesco_con40', '2nd', 'worst', 'QB', '.', 'DEFINITELY', 'Tony', 'Romo', '.', 'The', 'man', 'who', 'likes', 'to', 'share', 'the', 'ball', 'with', 'everyone', '.', 'Including', 'the', 'other', 'team', '.']

Example 4 Token indices: ['#', 'Thailand', 'Washington', '-', 'US', 'President', 'Barack', 'Obama', 'vowed', 'Wednesday', 'as', 'he', 'visited', 'storm-ravaged', 'New', 'Jersey', 'shore', 'to', '...', 'http', ':', '://t.co/XzL4LFhs']

Example 5 Token indices: ['Did', 'y\u0029all', 'hear', 'what', 'Tony', 'Romo', 'dressed', 'up', 'as', 'for', 'Halloween', '?', 'A', 'Giants', 'quarterback', '!', 'Cause', 'that\u0029s', 'all', 'he', 'could', 'throw', 'to', 'sunday', 'night', '.']

Example 6 Token indices: ['Tim', 'Tebow', 'may', 'be', 'available', '!', 'Wow', 'Jerry', '\u002c', 'what', 'the', 'heck', 'you', 'waiting', 'for', '!', 'http', ':', '://t.co/a729F8L4']

Example 7 Token indices: ['@', 'mariakaykay', 'aga', 'tayo', 'ah', '.', ':', ' '), 'Good', 'night\u002c', 'Ces', '.', 'Love', 'you', '!', '>', ':', 'D', '<']

Example 8 Token indices: ['Tina', 'Fey', '&', 'Amy', 'Poehler', 'are', 'hosting', 'the', 'Golden', 'Globe', 'awards', 'on', 'January', '13', '.', 'What', 'do', 'you', 'think', '?']

Example 9 Token indices: ['Lunch', 'from', 'my', 'new', 'Lil', 'spot', '...', 'THE', 'COTTON', 'BOWL', '...', 'pretty', 'good', '#', '1st', '#', 'time', '#', 'will', 'be', 'going', 'back', '#', 'http', ':', '://t.co/Dbj8xLZ']

Example 10 Token indices: ['SNC', 'Halloween', 'Pr', '.', 'Pumped', '.', 'Let\u0029s', 'work', 'it', 'for', 'Sunday', '...', 'Packers', 'vs', '...', 'who', 'knows', 'or', 'caresn', '.', '#', 'SNC', '#', 'cheerpracticeonhalloween']

Figure 3: Tokenized data for Semeval2017A

Example 1 Token indices: ['the', 'rock', 'is', 'destined', 'to', 'be', 'the', '21st', 'century', '"s', 'new', '', 'conan', '', 'and', 'that', 'he', 's', 'going', 'to', 'make', 'a', 'splash', 'even', 'greater', 'than', 'arnold', 'schwarzenegger', ',', 'jean-claud', 'van', 'damme', 'or', 'steven', 'segal', ',']

Example 2 Token indices: ['the', 'gorgeously', 'elaborate', 'continuation', 'of', '', 'the', 'lord', 'of', 'the', 'rings', '', 'trilogy', 'is', 'so', 'huge', 'that', 'a', 'column', 'of', 'words', 'can', 'not', 'adequately', 'describe', 'co-writer/director', 'peter', 'jackson', '"s', 'expanded', 'vision', 'of', 'j.', 'r.', 'f.', 'tolkien', '"s', 'middle-earth', '.']

Example 3 Token indices: ['effective', 'but', 'too- tepid', 'biopic']

Example 4 Token indices: ['if', 'you', 'sometimes', 'like', 'to', 'go', 'to', 'the', 'movies', 'to', 'have', 'fun', ',', 'wasabi', 'is', 'a', 'good', 'place', 'to', 'start', '.']

Example 5 Token indices: ['emerges', 'as', 'something', 'rare', ',', 'an', 'issue', 'movie', 'that', '"s', 'so', 'honest', 'and', 'keenly', 'observed', 'that', 'it', 'does', "n't", 'feel', 'like', 'one', '.']

Example 6 Token indices: ['the', 'film', 'provides', 'some', 'great', 'insight', 'into', 'the', 'neurotic', 'mindset', 'of', 'all', 'comics', '-', 'even', 'those', 'who', 'have', 'reached', 'the', 'absolute', 'top', 'of', 'the', 'game', '.']

Example 7 Token indices: ['offers', 'that', 'rare', 'combination', 'of', 'entertainment', 'and', 'education', '.']

Example 8 Token indices: ['perhaps', 'no', 'picture', 'ever', 'made', 'has', 'more', 'literally', 'showed', 'that', 'the', 'road', 'to', 'hell', 'is', 'paved', 'with', 'good', 'intentions', '.']

Example 9 Token indices: ['steers', 'in', 'a', 'snappy', 'screenplay', 'that', 'curls', 'at', 'the', 'edges', ';', 'it', '"s', 'so', 'clever', 'you', 'want', 'to', 'hate', 'it', 'but', 'he', 'somehow', 'pulls', 'it', 'off', '.']

Example 10 Token indices: ['take', 'care', 'of', 'my', 'cat', 'offers', 'a', 'refreshingly', 'different', 'slice', 'of', 'asian', 'cinema', '.']

Figure 4: Tokenized data for MR

1.3 Κωδικοποίηση Παραδειγμάτων (Λέξεων)

Στο τελικό στάδιο της προεπεξεργασίας, κάθε πρόταση του συνόλου δεδομένων μετατρέπεται σε μορφή κατάλληλη για εκπαίδευση ενός νευρωνικού δικτύου. Αρχικά, κάθε λέξη (token) χαρακτηρίζεται σε έναν ακέραιο αριθμό μέσω ενός λεξικού (word2idx) που δημιουργείται από τα προεκπαιδευμένα word embeddings. Αν μια λέξη δεν υπάρχει στο λεξικό, αντιστοιχίζεται σε ειδικό δείκτη για άγνωστες λέξεις (<unk>). Στη συνέχεια, για να εξασφαλιστεί ομοιομορφία στο μήκος των προτάσεων, εφαρμόζεται είτε συμπλήρωση με μηδενικά (zero-padding) σε προτάσεις μικρότερες του μέγιστου επιτρεπτού μήκους, είτε αποκοπή (truncation) για μεγαλύτερες. Η υλοποίηση αυτής της διαδικασίας πραγματοποιείται μέσω της μεθόδου `__getitem__` της κλάσης `SentenceDataset`, η οποία επιστρέφει για κάθε παράδειγμα: τη κωδικοποιημένη πρόταση, την αριθμητική ετικέτα της και το πραγματικό της μήκος πριν το padding. Η μέθοδος `__getitem__` παρουσιάζεται παρακάτω:

```
# EX3
example = self.tokenized_data[index]
max_length = 10
```

```

# mapping of tokens to ids using word2idx
example = [self.word2idx.get(token, self.word2idx['<unk>']) for token in example]
label = self.labels[index]
length = len(example)

# zero padding using maximum length
# or truncation in case of a larger example
if length < max_length:
    example += [0] * (max_length - length)
else:
    example = example[:max_length]

return np.array(example), label, length

```

Ζητούμενο 3: Παρακάτω παρουσιάζονται πέντε ενδεικτικά παραδείγματα της αρχικής μορφής και όπως τα επιστρέφει η κλάση **SentenceDataset** για καθένα από τα δύο dataset.

```

dataitem = "Won the match #getin . Plus\u002c tomorrow is a very busy day\u002c with Awareness Day\u2019s and debates. Gulp. Debates...", label = "1"
Return values:
example = [ 400001 1 551 2750 400001 3 400001 4003 15 8]
label = 1
length = 23

dataitem = "Some areas of New England could see the first flakes of the season Tuesday.", label = "1"
Return values:
example = [ 400001 620 4 400001 400001 95 254 1 59 26301]
label = 1
length = 15

dataitem = "@francesco_con40 2nd worst QB. DEFINITELY Tony Romo. The man who likes to share the ball with everyone. Including the other team.", label = "0"
Return values:
example = [ 17528 400001 2767 1608 400001 3 400001 400001 400001 3]
label = 0
length = 26

dataitem = "#Thailand Washington - US President Barack Obama vowed Wednesday as he visited storm-ravaged New Jersey shore to... http://t.co/Xz14LFhs", label = "1"
Return values:
example = [ 2750 400001 400001 12 400001 400001 400001 400001 3417 400001]
label = 1
length = 22

dataitem = "Did y\u2019all hear what Tony Romo dressed up as for Halloween? A Giants quarterback! Cause that\u2019s all he could throw to sunday night.", label = "1"
Return values:
example = [ 400001 400001 2272 103 400001 400001 4958 61 20 11]
label = 1
length = 26

```

Figure 5: Example data for Semeval2017A

```

dataitem = "the rock is destined to be the 21st century's new " conan " and that he's going to make a splash even greater than arnold schwarzenegger , jean-claud van damme or steven segal .", label = "1"
Return values:
example = [ 1 1138 15 10454 5 31 1 5034 590 10]
label = 1
length = 36

dataitem = "the gorgeously elaborate continuation of " the lord of the rings " trilogy is so huge that a column of words cannot adequately describe co-writer/director peter jackson's expanded vision of j . r . r . tolkien's middle-earth .", label = "1"
Return values:
example = [ 1 78616 5135 10117 4 29 1 2371 4 1]
label = 1
length = 42

dataitem = "effective but too-tepid biopic", label = "1"
Return values:
example = [ 2038 35 400001 34277 0 0 0 0 0]
label = 1
length = 4

dataitem = "if you sometimes like to go to the movies to have fun , wasabi is a good place to start .", label = "1"
Return values:
example = [ 84 82 1072 118 5 243 5 1 2460 5]
label = 1
length = 21

dataitem = "emerges as something rare , an issue movie that's so honest and keenly observed that it doesn't feel like one .", label = "1"
Return values:
example = [ 12398 20 646 2349 2 30 496 1006 13 10]
label = 1
length = 23

```

Figure 6: Example data for MR

2 Μοντέλο

Σε αυτό το στάδιο σχεδιάζουμε ένα νευρωνικό δίκτυο το οποίο θα δημιουργεί διανυσματική αναπαράσταση για κάθε όρο μέσω ενός *embedding layer*, θα παράγει ενιαία αναπαράσταση για όλο το κείμενο, και θα προβλέπει την κατηγορία του κειμένου βάσει αυτής της αναπαράστασης. Η υλοποίηση γίνεται στην κλάση *BaselineDNN* στο αρχείο *models.py*, όπου στη μέθοδο *__init__()* δηλώνονται τα *layers* και στην *forward()* καθορίζεται η ροή των δεδομένων προς την έξοδο του μοντέλου.

2.1 Embedding Layer

Στο στάδιο αυτό, χρησιμοποιούμε ένα *embedding layer* για να προβάλλει κάθε λέξη ενός κειμένου σε έναν συνεχές διανυσματικό χώρο (*embedding space*). Σε αυτόν τον χώρο, λέξεις που εμφανίζονται σε παρόμοια συμφραζόμενα βρίσκονται κοντά μεταξύ τους. Το διάνυσμα που αντιστοιχεί σε κάθε λέξη ονομάζεται *word embedding* και αποτελεί αναπαράσταση των σημασιολογικών και συντακτικών χαρακτηριστικών της λέξης. Το *embedding layer* μπορεί να αρχικοποιηθεί είτε τυχαία είτε με προεκπαιδευμένα *word embeddings*. Για το σκοπό αυτό χρησιμοποιείται η συνάρτηση *load_word_vectors*, η οποία επιστρέφει έναν πίνακα διαστάσεων (*num_embeddings*, *emb_dim*) με τα προεκπαιδευμένα βάρη.

Υλοποιούνται τα εξής:

- Δημιουργία του *embedding layer*.
- Αρχικοποίηση των βαρών με τα προεκπαιδευμένα *word embeddings*.
- Επιλογή για πάγωμα ή όχι των βαρών κατά την εκπαίδευση του μοντέλου.

Απαντήσεις:

- **Γιατί αρχικοποιούμε το *embedding layer* με προ-εκπαιδευμένα *word embeddings*;**
Τα προεκπαιδευμένα *embeddings* έχουν εκπαιδευτεί πάνω σε τεράστια σώματα κειμένου και έχουν μάθει πλούσιες σημασιολογικές και συντακτικές σχέσεις μεταξύ των λέξεων. Αρχικοποιώντας το μοντέλο μας με αυτά, του προσδίδουμε εξαρχής καλύτερη γενίκευση, γεγονός που επιταχύνει τη σύγκλιση κατά την εκπαίδευση και βελτιώνει την απόδοσή του, ιδιαίτερα όταν το δικό μας *dataset* είναι μικρό ή περιορισμένο σε ποικιλία.
- **Γιατί κρατάμε παγωμένα τα βάρη του *embedding layer* κατά την εκπαίδευση;**
Το πάγωμα των βαρών (*freeze*) σημαίνει ότι δεν επιτρέπουμε την ενημέρωση των *word embeddings* κατά την εκπαίδευση. Αυτό είναι ιδιαίτερα σημαντικό όταν χρησιμοποιούμε μικρότερο *dataset*, καθώς μειώνει τον κίνδυνο υπερπροσαρμογής (*overfitting*) στα δεδομένα μας. Επιπλέον, διατηρεί τη γενικότητα των *embeddings*, που έχουν μάθει καθολικές γλωσσικές σχέσεις. Εναλλακτικά, μπορούμε να επιτρέψουμε την ενημέρωση των βαρών (*fine-tuning*), εφόσον θέλουμε το μοντέλο να προσαρμοστεί περισσότερο στα χαρακτηριστικά του συγκεκριμένου προβλήματος.

Ζητούμενο 4: Παρακάτω παρουσιάζονται τα σημεία του κώδικα που προσθέσαμε στο αρχείο **models.py**, έτσι ώστε να υλοποιήσουμε τα παραπάνω ζητούμενα.

```
def __init__(self, output_size, embeddings, trainable_emb=False):
    """
    Args:
        output_size(int): the number of classes
        embeddings(bool): the 2D matrix with the pretrained embeddings
        trainable_emb(bool): train (finetune) or freeze the weights
                           the embedding layer
    """
```

```

super(BaselineDNN, self).__init__()

# EX4
# 1 - define the embedding layer
num_embeddings = len(embeddings)
dim = len(embeddings[0])
self.embeddings = nn.Embedding(num_embeddings, dim)
self.output_size = output_size

# 2 - initialize the weights of our Embedding layer
# from the pretrained word embeddings
# 3 - define if the embedding layer will be frozen or finetuned
if not trainable_emb:
    self.embeddings = self.embeddings.from_pretrained(torch.Tensor(embeddings),
                                                         freeze = True)

```

2.2 Output Layer(s)

Στο συγκεκριμένο στάδιο γίνεται προβολή των αναπαραστάσεων των κειμένων στον χώρο των κλάσεων, ώστε να προκύψει η τελική πρόβλεψη του μοντέλου. Αξίζει να σημειωθεί πως στο δίκτυο `BaselineDNN` έχει οριστεί `hidden_size = 1000`, δηλαδή το ενδιάμεσο επίπεδο του μοντέλου μετασχηματίζει τις αναπαραστάσεις σε διανύσματα 1000 διαστάσεων. Αν, για παράδειγμα, το πρόβλημα της ταξινόμησης έχει 3 κατηγορίες (όπως στο `Semeval2017A`), τότε το τελικό επίπεδο θα πρέπει να πραγματοποιεί προβολή:

$$R^{1000} \rightarrow R^3$$

Αντίστοιχα, για το MR όπου υπάρχουν 2 κατηγορίες:

$$R^{1000} \rightarrow R^2$$

Υλοποιούνται τα εξής:

- Δημιουργία ενός ενδιάμεσου layer με μία μη γραμμική συνάρτηση ενεργοποίησης (όπως ReLU, tanh), το οποίο θα μάθει έναν μη γραμμικό μετασχηματισμό της αναπαράστασης κάθε παραδείγματος.
- Δημιουργία του τελικού επιπέδου εξόδου, το οποίο προβάλλει τις αναπαραστάσεις των παραδειγμάτων στον χώρο των κλάσεων, δηλαδή στον επιθυμητό αριθμό κατηγοριών.

Απαντήσεις:

- **Γιατί βάζουμε μη γραμμική συνάρτηση ενεργοποίησης στο προτελευταίο layer;**
Η χρήση μιας μη γραμμικής συνάρτησης ενεργοποίησης, όπως η ReLU ή η tanh, επιτρέπει στο μοντέλο να μάθει πιο πολύπλοκους και μη γραμμικούς συσχετισμούς στα δεδομένα. Ένα νευρωνικό δίκτυο χωρίς μη γραμμικότητα θα ισοδυναμούσε με έναν απλό γραμμικό μετασχηματισμό, περιορίζοντας σημαντικά την εκφραστικότητά του.
- **Τι διαφορά θα είχε αν είχαμε 2 ή περισσότερους γραμμικούς μετασχηματισμούς στη σειρά;**
Αν εφαρμόζαμε δύο ή περισσότερους διαδοχικούς γραμμικούς μετασχηματισμούς χωρίς ενδιάμεση μη γραμμικότητα, το αποτέλεσμα θα μπορούσε να συγχωνευτεί σε έναν μόνο γραμμικό μετασχηματισμό. Επομένως, χωρίς μη γραμμικές συναρτήσεις ανάμεσά τους, η προσθήκη πολλών γραμμικών layers δεν προσθέτει υπολογιστική ισχύ στο μοντέλο. Η ουσιαστική διαφοροποίηση επιτυγχάνεται μόνο με την εισαγωγή μη γραμμικότητας μεταξύ των επιπέδων.

Ζητούμενο 5: Παρακάτω παρουσιάζονται τα σημεία του κώδικα που προσθέσαμε στο αρχείο **models.py** και συγκεκριμένα στην συνάρτηση `__init__`, έτσι ώστε να υλοποιήσουμε τα παραπάνω ζητούμενα για το output layer. Αξίζει να σημειώσουμε ότι για το μη γραμμικό μετασχηματισμό χρησιμοποιούμε τη συνάρτηση ενεργοποίησης ReLU.

```
# 4 - define a non-linear transformation of the representations
# EX5
self.linear = nn.Linear(dim, 1000)
self.relu = nn.ReLU()

# 5 - define the final Linear layer which maps
# the representations to the classes
# EX5
self.output = nn.Linear(1000, output_size)
```

2.3 Forward Pass

Αφού σχεδιάσαμε τα απαραίτητα επίπεδα (layers) του μοντέλου, το τελικό βήμα είναι να καθοριστεί η διαδικασία με την οποία τα δεδομένα εισόδου θα μετασχηματιστούν στις αντίστοιχες εξόδους (προβλέψεις). Αυτό υλοποιείται στη μέθοδο `forward()` του μοντέλου. Θεωρούμε ότι η είσοδος είναι ένα mini-batch προτάσεων με διαστάσεις (batch_size, max_length). Η διαδικασία περιλαμβάνει τα εξής βασικά στάδια:

- **Ενσωμάτωση λέξεων:** Κάθε λέξη κάθε πρότασης περνά από το *embedding layer* που δημιουργήθηκε, ώστε να προβάλλεται σε έναν συνεχές διανυσματικό χώρο. Έτσι, η είσοδος (batch_size, max_length) μετατρέπεται σε (batch_size, max_length, emb_dim).
- **Δημιουργία αναπαράστασης πρότασης:** Καθώς οι προτάσεις έχουν μεταβλητό μήκος, εφαρμόζεται μία τεχνική σύνοψης ώστε να παραχθεί μία ενιαία αναπαράσταση για κάθε πρόταση. Χρησιμοποιείται η μέση τιμή (*mean pooling*) των word embeddings κάθε πρότασης, αγνοώντας τις θέσεις που έχουν προκύψει από padding. Έτσι, όλες οι προτάσεις αποκτούν μία ενιαία διανυσματική αναπαράσταση διάστασης ίσης με emb_dim.
- **Μη γραμμικός μετασχηματισμός:** Η αναπαράσταση κάθε πρότασης περνά από ένα ενδιάμεσο επίπεδο (Linear + ReLU), ώστε να μετασχηματιστεί σε μια πιο αφηρημένη, υψηλότερης διάστασης αναπαράσταση.
- **Τελική προβολή στις κλάσεις:** Η τελική προβολή των αναπαραστάσεων γίνεται μέσω ενός Linear layer, το οποίο εξάγει τις προβλέψεις του μοντέλου για κάθε κατηγορία.

Απαντήσεις:

- **Διανυσματική ερμηνεία της μέσης τιμής:** Αν θεωρήσουμε ότι κάθε διάσταση του embedding χώρου αντιστοιχεί σε μία αφηρημένη έννοια (π.χ. θετικότητα, ενέργεια, χρονικότητα), τότε η μέση τιμή των διανυσμάτων μιας πρότασης μπορεί να ερμηνευθεί ως ένας «μέσος όρος νοήματος» της πρότασης. Δηλαδή, μία συνδυαστική αναπαράσταση που βρίσκεται κοντά στο κέντρο βάρους των εννοιών που εκφράζουν οι λέξεις της πρότασης.
- **Αδυναμίες της προσέγγισης:** Η συγκεκριμένη μέθοδος (mean pooling) αγνοεί τη σειρά των λέξεων και χάνει πληροφορία για τη σύνταξη και το πλαίσιο χρήσης. Επίσης, όλες οι λέξεις συμβάλλουν ισότιμα, ανεξάρτητα από τη σημασία τους, γεγονός που ενδέχεται να επηρεάσει αρνητικά την απόδοση του μοντέλου σε πιο απαιτητικά καθήκοντα κατανόησης.

Ζητούμενο 6: Παρακάτω παρουσιάζονται τα σημεία του κώδικα που προσθέσαμε στο αρχείο **models.py** και συγκεκριμένα στη μέθοδο **forward**.

```

def forward(self, x, lengths):
    """
    This is the heart of the model.
    This function, defines how the data passes through the network.

    Returns: the logits for each class

    """

    # 1 - embed the words, using the embedding layer
    # EX6
    embeddings = self.embeddings(x)

    # 2 - construct a sentence representation out of the word embeddings
    # EX6
    # calculate the means
    representations = torch.sum(embeddings, dim=1)
    for i in range(lengths.shape[0]): # skip zeros in mean calculation
        representations[i] = representations[i] / lengths[i]

    # 3 - transform the representations to new ones.
    # EX6
    representations = self.relu(self.linear(representations))

    # 4 - project the representations to classes using a linear layer
    # EX6
    logits = self.output(representations)

    return logits

```

3 Διαδικασία Εκπαίδευσης

Σε αυτό το βήμα υλοποιούμε τη διαδικασία εκπαίδευσης του δικτύου μας. Αρχικά, οργανώνουμε τα δεδομένα σε mini-batches μέσω του DataLoader της PyTorch. Έπειτα, για κάθε batch, κάνουμε forward pass στο μοντέλο, υπολογίζουμε τη συνάρτηση απώλειας και εφαρμόζουμε backpropagation για να ενημερώσουμε τα βάρη του μοντέλου με τη βοήθεια του optimizer (Adam). Η διαδικασία αυτή επαναλαμβάνεται για προκαθορισμένο αριθμό εποχών μέχρι να επιτευχθεί σύγκλιση ή να ενεργοποιηθεί κάποιος μηχανισμός early stopping.

3.1 Φόρτωση Παραδειγμάτων (DataLoaders)

Αφού υλοποιήσαμε την κλάση SentenceDataset για τη μετατροπή των παραδειγμάτων σε κατάλληλη μορφή εισόδου, το επόμενο βήμα είναι να οργανώσουμε τα δεδομένα για την εκπαίδευση του νευρωνικού δικτύου. Για τον σκοπό αυτό αξιοποιείται η κλάση DataLoader της βιβλιοθήκης PyTorch, η οποία αναλαμβάνει τη δημιουργία των mini-batches και τη διαχείριση της ροής των δεδομένων κατά την εκπαίδευση. Για κάθε dataset δημιουργείται ένας DataLoader, ο οποίος φορτώνει τα δεδομένα σε μικρά πακέτα (mini-batches) σταθερού μεγέθους (εδώ έχουμε 128 παραδείγματα ανά batch), επιτρέποντας στο μοντέλο να υπολογίζει την απώλεια και να ενημερώνει τα βάρη του σταδιακά.

Απαντήσεις:

- Τι συνέπειες έχουν τα μικρά και μεγάλα mini-batches στην εκπαίδευση των μοντέλων;

Τα μικρά mini-batches οδηγούν σε πιο ασταθή αλλά ταχύτερη εκπαίδευση, καθώς παράγουν πιο "θορυβώδεις" εκτιμήσεις του gradient. Αυτό μπορεί να βοηθήσει στην αποφυγή τοπικών ελαχίστων, αλλά συχνά απαιτεί περισσότερες εποχές για να επιτευχθεί σύγκλιση. Αντίθετα, τα μεγάλα mini-batches προσφέρουν πιο

σταθερούς υπολογισμούς του gradient και συχνά οδηγούν σε πιο ομαλή σύγκλιση, αλλά με υψηλότερες απαιτήσεις σε μνήμη και μικρότερη ικανότητα γενίκευσης (generalization).

- **Γιατί ανακατεύουμε τη σειρά των mini-batches σε κάθε εποχή;**

Η ανακάτεψη (shuffling) των παραδειγμάτων σε κάθε εποχή αποτρέπει το μοντέλο από το να "μάθει" τη σειρά των δεδομένων, γεγονός που θα μπορούσε να οδηγήσει σε υπερεκπαίδευση (overfitting) ή σε κακή γενίκευση. Επιπλέον, διασφαλίζει ότι τα mini-batches κάθε εποχής περιέχουν διαφορετικό συνδυασμό παραδειγμάτων, ενισχύοντας τη στατιστική ποικιλία της εκπαίδευσης.

Ζητούμενο 7: Παρακάτω παρουσιάζονται τα σημεία του κώδικα που προσθέσαμε στο αρχείο **main.py** για τους **DataLoaders**.

```
# EX7 - Define our PyTorch-based DataLoader
train_loader = DataLoader(train_set, batch_size= BATCH_SIZE, shuffle=True) # EX7
test_loader = DataLoader(test_set, batch_size=BATCH_SIZE, shuffle=False) #EX7
```

Όπως αναφέραμε και παραπάνω, `BATCH_SIZE = 128` και επίσης στο `test_loader` επιλέγουμε `shuffle = False`, καθώς δεν επηρεάζει το στάδιο του training και θέλουμε να έχουμε σταθερά σημεία αναφοράς για το testing.

3.2 Βελτιστοποίηση

Για την εκπαίδευση και βελτιστοποίηση του νευρωνικού μοντέλου είναι απαραίτητο να καθοριστούν τρία βασικά στοιχεία: η συνάρτηση απώλειας (loss function), οι παράμετροι προς εκπαίδευση και ο αλγόριθμος βελτιστοποίησης.

- **Συνάρτηση απώλειας (Loss function):**

Για την επιλογή του κατάλληλου κριτηρίου επειδή το πρόβλημα περιλαμβάνει περισσότερες από 2 κατηγορίες (MR - 2, Semeval2017A - 3), χρησιμοποιούμε τη `CrossEntropyLoss`. Στην περίπτωση αυτή, απαιτείται το `output layer` του μοντέλου να έχει τόσες εξόδους όσες οι κλάσεις του προβλήματος.

- **Επιλογή παραμέτρων προς εκπαίδευση:**

Είναι σημαντικό να καθοριστεί ποιες παράμετροι του μοντέλου θα βελτιστοποιηθούν. Συνήθως, δεν επιθυμούμε να εκπαιδεύσουμε όλα τα επίπεδα — για παράδειγμα, τα βάρη του `embedding layer` παραμένουν σταθερά (frozen) αφού βασίζονται σε προεκπαιδευμένα embeddings. Γι' αυτό, γίνεται φιλτράρισμα των παραμέτρων βάσει της ιδιότητας `requires_grad = True`.

- **Αλγόριθμος βελτιστοποίησης (Optimizer):**

Ο optimizer καθορίζει τον τρόπο με τον οποίο ενημερώνονται τα βάρη του μοντέλου σε κάθε βήμα. Στην πράξη γίνεται η χρήση του αλγορίθμου Adam, καθώς προσφέρει ταχύτερη και πιο σταθερή σύγκλιση σε σχέση με τον απλό SGD.

Ζητούμενο 8: Παρακάτω παρουσιάζονται τα σημεία του κώδικα που προσθέσαμε στο αρχείο **main.py** για τα στοιχεία της βελτιστοποίησης.

```
#Define model
model = BaselineDNN(output_size= n_classes, # EX8
                    embeddings=embeddings,
                    trainable_emb=EMB_TRAINABLE)

# move the mode weight to cpu or gpu
model.to(DEVICE)
print(model)

#Loss Function
criterion = torch.nn.CrossEntropyLoss()

#Parameters and optimizer
```

```
parameters = filter(lambda p: p.requires_grad, model.parameters()) # EX8
optimizer = torch.optim.Adam(parameters, lr=1e-3) # 10x smaller
```

3.3 Εκπαίδευση

Το τελικό βήμα στην υλοποίηση του συστήματος είναι να οριστούν οι διαδικασίες για την εκπαίδευση και την αξιολόγηση του μοντέλου ανά mini-batch και ανά εποχή. Για τον σκοπό αυτό, υλοποιούνται οι συναρτήσεις `train_dataset` και `eval_dataset`. Η συνάρτηση `train_dataset` εκτελείται σε κάθε εποχή και είναι υπεύθυνη για την εκπαίδευση του μοντέλου. Συγκεκριμένα, για κάθε mini-batch δεδομένων: υπολογίζεται η έξοδος του μοντέλου μέσω forward pass, υπολογίζεται η συνάρτηση απώλειας (loss), εφαρμόζεται ο αλγόριθμος *backpropagation*, ώστε να υπολογιστούν τα gradients και ο optimizer ενημερώνει τα βάρη του δικτύου. Αντίστοιχα, η συνάρτηση `eval_dataset` καλείται στο τέλος κάθε εποχής και χρησιμοποιείται για την αξιολόγηση του μοντέλου στο σύνολο επικύρωσης ή στο test set, χωρίς να γίνεται ενημέρωση των βαρών. Η λειτουργία αυτή επιτρέπει την παρακολούθηση της απόδοσης του μοντέλου και τη χρήση τεχνικών όπως το *early stopping*, το οποίο όμως δεν θα χρησιμοποιήσουμε στη φάση της προπαρασκευής.

Ζητούμενο 9: Παρακάτω παρουσιάζονται οι υλοποιήσεις των συναρτήσεων `train_dataset` και `eval_dataset`.

```
def train_dataset(_epoch, dataloader, model, loss_function, optimizer):
    # IMPORTANT: switch to train mode
    # enable regularization layers, such as Dropout
    model.train()
    running_loss = 0.0

    # obtain the model's device ID
    device = next(model.parameters()).device

    for index, batch in enumerate(dataloader, 1):
        # get the inputs (batch)
        inputs, labels, lengths = batch

        # EX9
        # move the batch tensors to the right device
        inputs = inputs.to(device)
        labels = labels.to(device)
        lengths = lengths.to(device)

        # Step 1 - zero the gradients -Clear gradients before each batch!
        model.zero_grad()
        # Step 2 - forward pass: y' = model(x)
        if model.__class__.__name__ in ['BaselineDNN', 'LSTM']:
            outputs = model(inputs, lengths)
        else:
            outputs = model(inputs)

        # Step 3 - compute loss: L = loss_function(y, y')
        try:
            loss = loss_function(outputs, labels)
        except ValueError:
            # fix labels for 'BCEWithLogitsLoss' loss function
            bin_labels = torch.nn.functional.one_hot(labels.long(), num_classes=2)
            loss = loss_function(outputs, bin_labels.float())
```

```

        # Step 4 - backward pass: compute gradient wrt model parameters
        loss.backward()
        # Step 5 - update weights
        optimizer.step()

    running_loss += loss.data.item()

    # print statistics
    progress(loss=loss.data.item(),
             epoch=_epoch,
             batch=index,
             batch_size=dataloader.batch_size,
             dataset_size=len(dataloader.dataset))

    return running_loss / index

def eval_dataset(dataloader, model, loss_function):
    # IMPORTANT: switch to eval mode
    # disable regularization layers, such as Dropout
    model.eval()
    running_loss = 0.0
    y_pred = [] # the predicted labels
    y = [] # the gold labels

    # obtain the model's device ID
    device = next(model.parameters()).device

    # IMPORTANT: in evaluation mode, we don't want to keep the gradients
    # so we do everything under torch.no_grad()
    with torch.no_grad():
        for index, batch in enumerate(dataloader, 1):
            # get the inputs (batch)
            inputs, labels, lengths = batch

            # Step 1 - move the batch tensors to the right device
            inputs = inputs.to(device)
            labels = labels.to(device)
            lengths = lengths.to(device)

            # Step 2 - forward pass: y' = model(x)
            if model.__class__.__name__ in ['BaselineDNN', 'LSTM']:
                outputs = model(inputs, lengths)
            else:
                outputs = model(inputs)

            # Step 3 - compute loss (same logic as in training)
            try:
                loss = loss_function(outputs, labels)
            except ValueError:
                bin_labels = torch.nn.functional.one_hot(labels.long(), num_classes=2)
                loss = loss_function(outputs, bin_labels.float())

```

```

# Step 4 - make predictions (class = argmax of posteriors or threshold)
if outputs.shape[1] == 1: # binary case with single output neuron
    preds = (torch.sigmoid(outputs) > 0.5).long().squeeze()
else:
    preds = torch.argmax(outputs, dim=1)

# Step 5 - collect the predictions, gold labels and batch loss
y_pred.append(preds.cpu().numpy())
y.append(labels.cpu().numpy())

running_loss += loss.data.item()

return running_loss / index, (y_pred, y)

```

3.4 Αξιολόγηση

Αφού έχουμε ολοκληρώσει την ανάπτυξη του μοντέλου μας, το τελικό ζητούμενο είναι να αξιολογήσουμε τις επιδόσεις του. Για τον λόγο αυτό, κατά την εκπαίδευση επιλέγουμε σταθερό αριθμό εποχών, ίσο με 50 και για τα δύο datasets. Παράλληλα, για καθένα από τα datasets, θα υπολογίσουμε βασικές μετρικές όπως, accuracy, F1_score, και recall. Για κάθε μια από αυτές τις μετρικές θα εμφανίσουμε το διάγραμμα με τις αντίστοιχες καμπύλες για training και validation set.

Ζητούμενο 10: Παρακάτω παρουσιάζονται οι ζητούμενες γραφικές παραστάσεις.

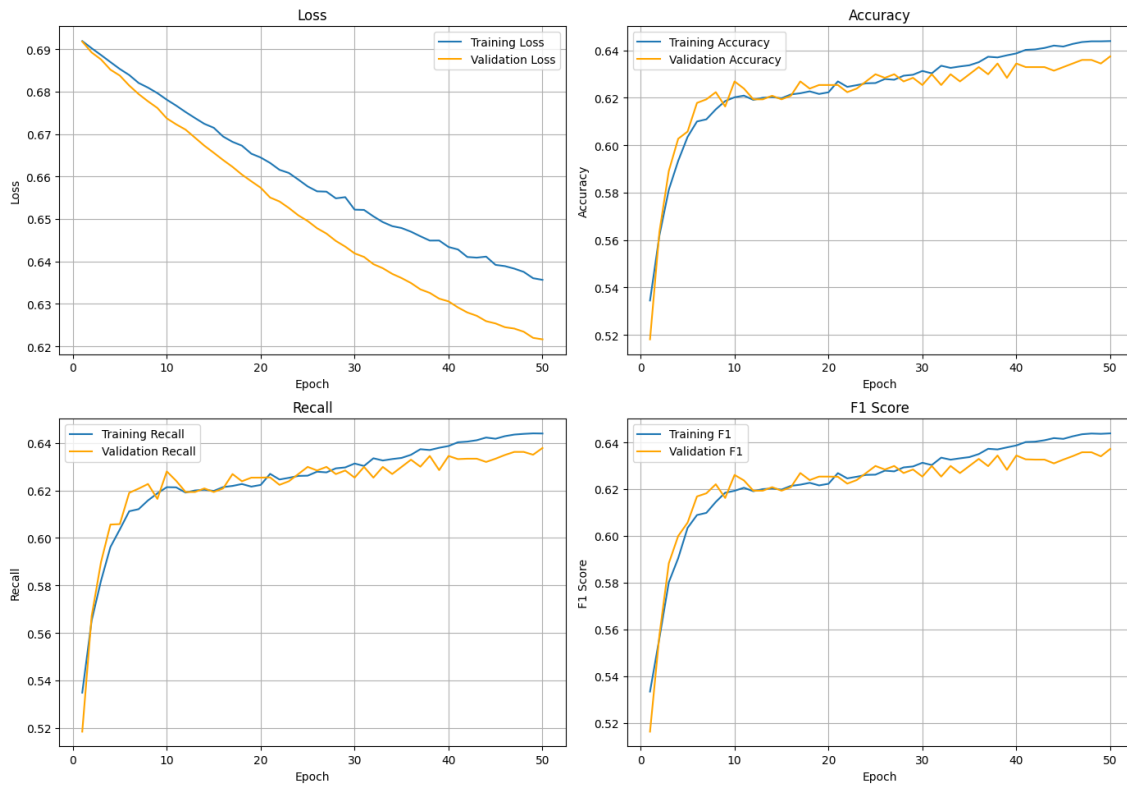


Figure 7: Semeval2017A: Glove-6B-50d Embeddings

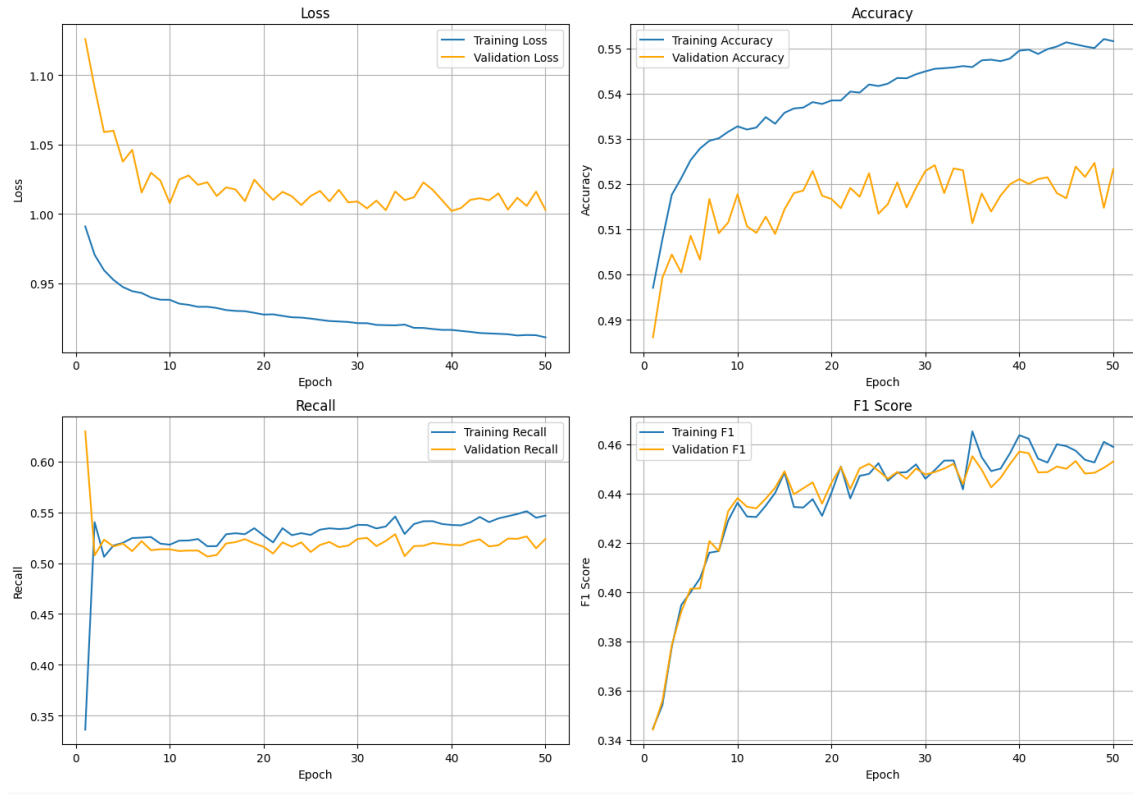


Figure 8: MR: Glove-6B-50d Embeddings

Τα παραπάνω διαγράμματα αποτυπώνουν την εξέλιξη της απόδοσης των μοντέλων κατά την εκπαίδευση και το validation, τόσο για το σύνολο δεδομένων Semeval2017A όσο και για το MR. Συνολικά, παρατηρούνται οι εξής βασικές τάσεις:

- **Στο σύνολο MR**, το μοντέλο παρουσιάζει σταθερή και συνεπή βελτίωση σε όλες τις μετρικές (Loss, Accuracy, Recall, F1), τόσο στο training όσο και στο validation set. Οι καμπύλες training και validation βρίσκονται πολύ κοντά, γεγονός που υποδεικνύει καλή γενίκευση και ισορροπημένη εκπαίδευση χωρίς υπερπροσαρμογή. Η χρήση της μέσης αναπαράστασης (mean pooling) φαίνεται να είναι επαρκής για αυτό το πρόβλημα.
- **Στο σύνολο Semeval2017A**, παρατηρείται σαφής απόσταση μεταξύ των καμπυλών training και validation, ιδιαίτερα στις μετρικές Loss και Accuracy. Αν και το μοντέλο βελτιώνεται συνεχώς στο training set, η απόδοση στο validation παραμένει σχεδόν σταθερή. Αυτό φανερώνει πιθανό *underfitting* ή περιορισμένη εκφραστικότητα του μοντέλου, λόγω απλοϊκής αναπαράστασης των προτάσεων. Είναι πιθανό το συγκεκριμένο dataset να απαιτεί πιο πλούσια ή δομικά ευαίσθητη προσέγγιση (όπως LSTM, attention ή transformer-based fine-tuning, που θα δούμε παρακάτω στο κύριο μέρος).
- **Σύγκριση recall και F1:** Στα περισσότερα σενάρια, οι καμπύλες Recall και F1 εμφανίζουν πιο ομαλή πορεία από την Accuracy, γεγονός που υποδηλώνει ότι το μοντέλο μπορεί να είναι πιο ικανό στο να αναγνωρίζει σωστά τις κατηγορίες χωρίς απαραίτητα υψηλή συνολική ακρίβεια.

Γενικά, η συμπεριφορά του μοντέλου στο MR dataset δείχνει ότι η αρχιτεκτονική μπορεί να γενικεύσει καλά όταν τα δεδομένα έχουν λιγότερη πολυπλοκότητα ή πιο ξεκάθαρη δομή. Αντιθέτως, η χαμηλή απόδοση στο Semeval2017A ενισχύει την ανάγκη για πιο σύνθετες τεχνικές αναπαράστασης ή fine-tuning προεκπαιδευμένων μοντέλων.

4 Κατηγοριοποίηση με χρήση LLM

Για την κατηγοριοποίηση προτάσεων με τη βοήθεια του **ChatGPT**, επιλέξαμε 20 αντιπροσωπευτικές προτάσεις από κάθε κατηγορία για κάθε dataset. Στη συνέχεια, ζητήσαμε από το μοντέλο να προσδιορίσει το συναίσθημα της κάθε πρότασης, σύμφωνα με τις κλάσεις που αντιστοιχούν σε κάθε dataset (π.χ. positive, negative, neutral). Το prompt που χρησιμοποιήσαμε ήταν το εξής:

#For MR

Can you classify each sentence as positive or negative and give an explanation for that too. But without using code.

#For Semeval2017A

Now do the same but also add neutral as a possible class.

#Specify the description

I want the explanation to include the reason,

for examplpe word choices etc, for the class you chose.

Στη συνέχεια πείραμε τα αποτελέσματα που μας έδωσε και τα αποθηκεύσαμε σε ένα csv αρχείο. Ενδεικτικά παρουσιάζονται κάποιες από τις προτάσεις που ταξινομήθηκαν.

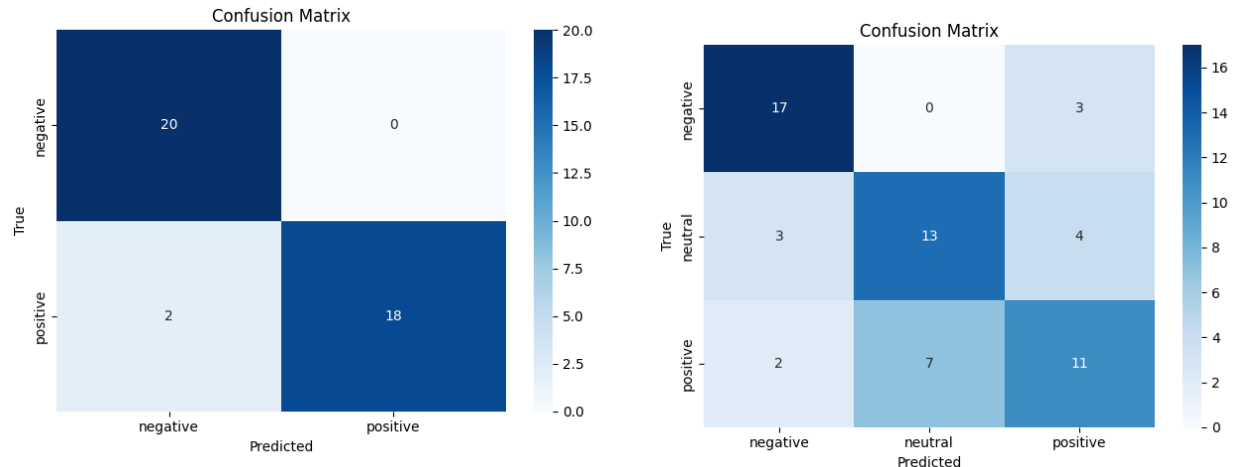
Πρόταση	Αληθινή Ετικέτα	Πρόβλεψη ChatGPT	Αιτιολόγηση
it's an unusual , thoughtful bio-drama with a rich subject and some fantastic moments and scenes .	positive	positive	Words like “thoughtful,” “rich subject,” and “fantastic moments” signal a clearly positive assessment.
ice age won't drop your jaw , but it will warm your heart , and i'm giving it a strong thumbs up .	positive	positive	“Warm your heart” and “strong thumbs up” are classic indicators of a positive review.
a movie where story is almost an afterthought amidst a swirl of colors and inexplicable events .	positive	negative	Phrases like “story is almost an afterthought” and “inexplicable events” suggest a lack of coherence — negative impression.
routine and rather silly.	negative	negative	Words like “routine” and “silly” convey a dismissive tone — not impressive.
lazy filmmaking , with the director taking a hands-off approach when he should have shaped the story to show us why it's compelling .	negative	negative	“Lazy filmmaking” and “hands-off approach” are direct critiques of the director's failure to engage.

Table 1: Ενδεικτικά παραδείγματα προτάσεων με τις προβλέψεις του ChatGPT για το MR και την αντίστοιχη αιτιολόγηση.

Πρόταση	Αληθινή Ετικέτα	Πρόβλεψη ChatGPT	Αιτιολόγηση
#ArianaGrande Ari By Ariana Grande 80% Full https://t.co/ylhCMETHHW #Singer #Actress https://t.co/lTrb1JQIEA	neutral	neutral	No emotional language or opinion; simply a promotional tweet with product name and hashtags. .
When will #BlackLivesMat- ter protest the black violence that kills nearly all blacks? https://t.co/CsztsCufp5 .	negative	positive	Supports democratic action and BLM hashtags — call to action is empowering and forward-looking.
My idols are #littlemix #justin- bieber #arianagrande.	positive	positive	“Idols” is a strongly positive word showing deep admiration for listed artists.
Iggy from the”Revolution Club” mad that Trump called #Chicago a ”sanctuary city” What????#BlackLivesMatter #MAGA https://t.co/Ip7q8snL45 .	negative	negative	Mocks protest reactions using rhetorical sarcasm (“What???”) and frames BLM as ridiculous.
@Billysmithjr33 of course it is relevant to #BlackLivesMatter, a black life was brutally taken away...	negative	positive	Reaffirms BLM relevance in the con- text of real loss (“a black life was brutally taken away”) — empa- thetic.

Table 2: Ενδεικτικά παραδείγματα προτάσεων με τις προβλέψεις του ChatGPT για το Semeval2017A και την αντίστοιχη αιτιολόγηση.

Παρακάτω παρουσιάζονται το confusion matrix για τις κατηγοριοποιήσεις του ChatGPT για καθένα από τα δύο datasets.



(a) Confusion matrix για το dataset MR (b) Confusion matrix για το dataset Semeval2017A

Figure 9: Σύγκριση confusion matrices μεταξύ των δύο datasets. Παρατηρούμε διαφορές στην απόδοση ανά κατηγορία, ειδικά στις ουδέτερες περιπτώσεις στο Semeval2017A.

Από τους πίνακες με τις αιτιολογήσεις παρατηρούμε πως για το MR έχουμε και πάλι καλύτερα αποτελέσματα και με την χρήση του ChatGPT, ενώ από τις αιτιολογήσεις που μας έδωσε μπορούμε να αντλήσουμε σημαντικές πληροφορίες για το ποιές λέξεις έχουν περισσότερη σημασία για την κατηγοριοποίηση κάθε πρότασης. Το σύνολο όλων των predictions και αιτιολογήσεων έχει αποθηκευτεί σε ένα csv αρχείο για το κάθε dataset.

Από τα confusion matrices που παρουσιάζονται παραπάνω, παρατηρούμε επίσης ότι το μοντέλο επιτυγχάνει σημαντικά καλύτερη επίδοση στο dataset MR, όπου οι προτάσεις είναι πιο σαφώς διατυπωμένες και διαχωρισμένες μεταξύ των δύο κατηγοριών (*positive* και *negative*). Αντιθέτως, στο dataset Semeval2017A, το οποίο περιλαμβάνει και την κατηγορία *neutral*, παρατηρείται μεγαλύτερη σύγχυση μεταξύ των κλάσεων, κυρίως λόγω της ύπαρξης προτάσεων με μεικτό ή ασαφές συναισθηματικό περιεχόμενο. Ειδικότερα, πολλές *neutral* προτάσεις ταξινομήθηκαν λανθασμένα ως *positive* ή *negative*, κάτι που υποδηλώνει τη δυσκολία του ChatGPT στο να εντοπίσει την απουσία έντονου συναισθηματικού φορτίου. Σε αυτό το σημείο αξίζει να σημειώσουμε ότι για τα misclassifications οι αιτιολογήσεις ήταν σε αρκετά σημεία εύστοχες και πιθανόν τα πραγματικά labels να μην είναι απολύτως σωστά, καθώς έχουν παρχθεί αυτοματοποιημένα και περιέχουν περίπολλα tweets, τα οποία δεν είναι εύκολο να ταξινομηθούν σε μια κατηγορία.

Κυρίως Μέρος Εργαστηρίου

5 Ερωτήματα

5.1 Ερώτημα 1

Συνενώνουμε τις αναπαραστάσεις που προκύπτουν από τον μέσο όρο (mean pooling) και του μεγίστου ανά διάσταση (max pooling) των word embeddings κάθε πρότασης.

Στο αρχικό μοντέλο της προπαρασκευής που έχουμε σχεδιάσει παράγουμε την διανυσματική αναπαράσταση κάθε λέξης:

$$e_i = \text{embed}(x_i)$$

Στην συνέχεια χρησιμοποιούμε τον μέσο όρο των embeddings για την αναπαράσταση του κειμένου και τέλος το διάνυσμα χαρακτηριστικών του κειμένου προκύπτει ως εξής από την αναπαράσταση αυτή (u):

$$r = \text{RELU}(Wu + b)$$

Αυτό που τροποποιούμε στο συγκεκριμένο ερώτημα είναι ο τρόπος που αναπαριστούμε το κείμενο, εδώ θεωρούμε την αναπαράστασή κάθε πρότασης ως την συνένωση του μέσου όρου και του μεγίστου ανά διάσταση των word embeddings κάθε πρότασης $E = \{e_1, e_2, \dots, e_N\}$:

$$u = [\text{mean}(E) || \text{max}(E)]$$

Παρακάτω φαίνεται ο ανανεωμένος κώδικας της μεθόδου forward του μοντέλου και η σύγκριση των αποτελεσμάτων του αρχικού μοντέλου και του μοντέλου που έχουμε παράξει στο ερώτημα αυτό. Εκτός από την αλλαγή στην forward μέθοδο έχουμε τροποποιήσει και την μέθοδο init ώστε να έχουμε τις σωστές διαστάσεις που προκύπτουν από την συνένωση.

Σημείωση: Τόσο σε αυτό το ερώτημα, όσο και στα παρακάτω χρησιμοποιούμε τις τεχνικές βελτιστοποίησης που έχουμε δοκιμάσει και στην προπαρασκευή, όπως το early stopping και κρατάμε το learning rate στην τιμή 10^{-4} , καθώς μετά από δοκιμές θεωρούμε ότι είναι η τιμή που δίνει τα πιο ξεκάθαρα και ομαλά αποτελέσματα. Τέλος, σημειώνουμε ότι για τα παρακάτω ερωτήματα, όπως και για αυτό έχουμε χρησιμοποιήσει στην ανάλυση μας το dataset MR.

```
#-----With mean and max pooling-----#
def forward(self, x, lengths):
    """
    This is the heart of the model.
    This function defines how the data passes through the network.

    Returns: the logits for each class
    """

    # 1 - embed the words, using the embedding layer
    embeddings = self.embeddings(x) # shape: [batch_size, seq_len, embedding_dim]

    # 2 - mean pooling
    sum_embeddings = torch.sum(embeddings, dim=1)
    mean_embeddings = sum_embeddings / lengths.unsqueeze(1)

    # 3 - max pooling
    # mask out padding tokens for max pooling
    mask = torch.arange(x.size(1)).unsqueeze(0).to(x.device) < lengths.unsqueeze(1)
    mask = mask.unsqueeze(2).expand_as(embeddings)
```

```

embeddings_masked = embeddings.masked_fill(~mask, float('-inf'))
max_embeddings = torch.max(embeddings_masked, dim=1).values

# 4 - concatenate mean and max pooling
representations = torch.cat([mean_embeddings, max_embeddings], dim=1)

# 5 - transform the representations
representations = self.relu(self.linear(representations))

# 6 - project the representations to classes
logits = self.output(representations)

return logits

```

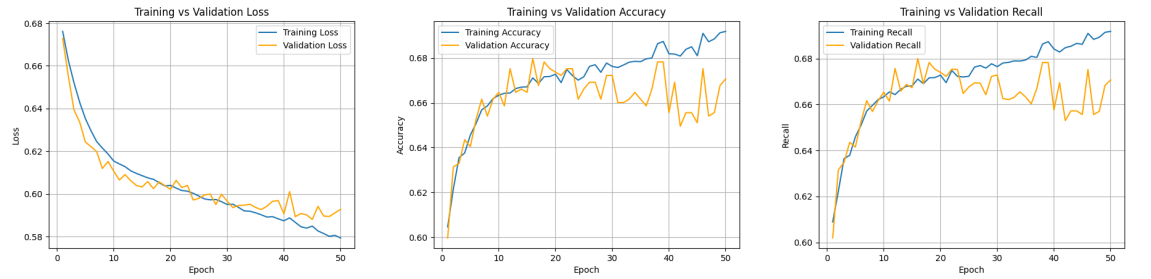


Figure 10: Results for Baseline model

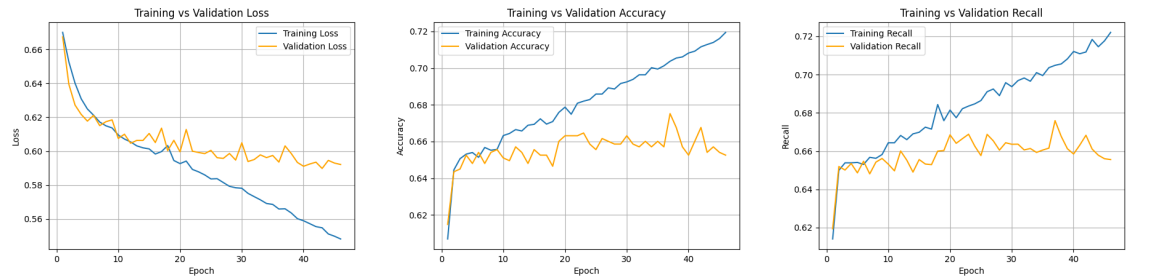


Figure 11: Results for (Baseline) Model using mean and max pooling

Η κύρια διαφορά που έχουμε με αυτή την αναπαράσταση των προτάσεων σε σχέση με την αρχική είναι ότι σε αυτή την περίπτωση δεν θεωρούμε την αναπαράσταση της πρότασης ως απλά των μέσο όρο των διανυσματικών αναπαραστάσεων των επιμέρους λέξεων, αλλά χρησιμοποιούμε και την πληροφορία της μέγιστης τιμής για κάθε διάσταση της αναπαράστασης για κάθε πρόταση.

Χρησιμοποιώντας το mean pooling για μια πρόταση ουσιαστικά παράγουμε την αναπαράσταση της πρότασης, χρησιμοποιώντας ισάξια τις αναπαραστάσεις όλων των λέξεων σε αυτήν την πρόταση. Όμως, όπως συμβαίνει και πραγματικά, μπορεί σε μια πρόταση να υπάρχουν λέξεις οι οποίες έχουν μεγαλύτερη σημασία από άλλες για το τελικό αποτέλεσμα. Σε αυτή την περίπτωση θεωρούμε το μέγιστο για κάθε διάσταση της αναπαράστασης. Καταφέρνουμε με αυτόν τον τρόπο να διαλέξουμε για κάθε διάσταση την μεγαλύτερη τιμή, η οποία και θα εκφράζει ένα χαρακτηριστικό το οποίο ξεχωρίζει για κάθε πρόταση. Τελικά, δηλαδή χρησιμοποιούμε μια εκτίμηση για όλα τα χαρακτηριστικά των λέξεων από τον μέσο όρο, αλλά και μια έκφραση των σημαντικότερων χαρακτηριστικών από το max pooling.

Στα παραπάνω αποτελέσματα δεν παρατηρούμε πολύ μεγάλες διαφορές, όμως φαίνεται ακόμα και με μικρές διαφορές, τα αποτελέσματα για την χρήση και max pooling να είναι λίγο καλύτερα στο training set. Αυτό σημαίνει ότι το μοντέλο μαθαίνει πιο σημαντικές πληροφορίες από τα δεδομένα εκπαίδευσης.

Σημειώνουμε για το παρακάτω αποτέλεσμα και τα αποτελέσματα των υπόλοιπων ερωτημάτων ότι όπου χρησιμοποιείται early stopping δεν παρουσιάζονται τα αποτελέσματα του τελευταίου εκπαιδευμένου μοντέλου, αλλά του καλύτερου (αυτό που προκύπτει όσα βήματα πριν όσα είναι η μεταβλητή patience και φαίνεται ξεκάθαρα καθώς ο κώδικας μας τυπώνει "best model").

Συγκεκριμένα έχουμε τα παρακάτω τελικά αποτελέσματα:

```
My test loss is : 0.5855833739042282
Accuracy for train: 0.69225
Accuracy for test: 0.6646525679758308
F1 score for train: 0.6913247265757061
F1 score for test: 0.663543956043956
Recall score for train: 0.6938409984260346
Recall score for test: 0.666851651451668
```

Figure 12: Metrics for baseline model (MR)

```
My train loss is : 0.5584768378545367
My test loss is : 0.5983456075191498
Accuracy for train: 0.708375
Accuracy for test: 0.6601208459214502
F1 score for train: 0.7083557469223554
F1 score for test: 0.6600580149396458
Recall score for train: 0.7083561812425858
Recall score for test: 0.6602393131165509
```

Figure 13: Metrics for baseline model with mean and max pooling (MR)

5.2 Ερώτημα 2

Σε αυτό το ερώτημα θα χρησιμοποιήσουμε ένα LSTM (Long Short Term Memory network) για να κωδικοποιήσουμε την πρόταση. Το LSTM θα διαβάζει τα embeddings που έχουμε δημιουργήσει για κάθε λέξη και θα παράγει για κάθε λέξη μία νέα αναπαράσταση, η οποία και βασίζεται και στα συμφραζόμενα της λέξης. Χωρίζουμε τα δεδομένα σε train και validation sets και χρησιμοποιούμε early stopper για να διακόπτουμε την διαδικασία της εκπαίδευσης όταν το validation loss αυξάνεται για 5 εποχές. Η αναπαράσταση του κειμένου είναι η τελευταία έξοδος του LSTM. Παρακάτω παρουσιάζουμε τα αποτελέσματα με την χρήση απλού LSTM και με την χρήση αμφίδρομου LSTM.

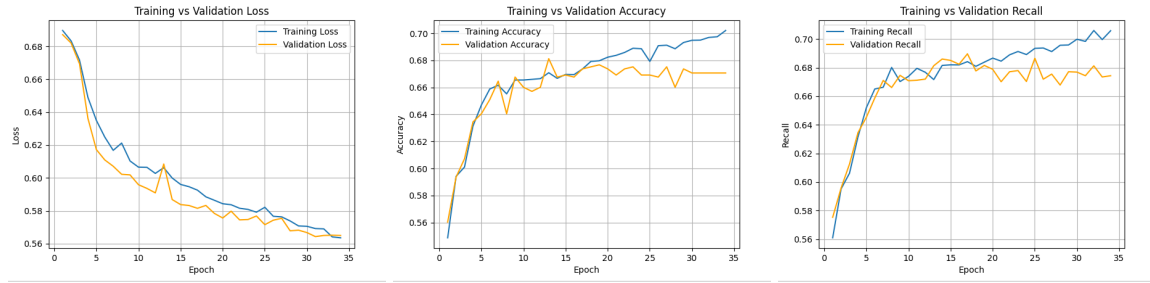


Figure 14: Results using LSTM

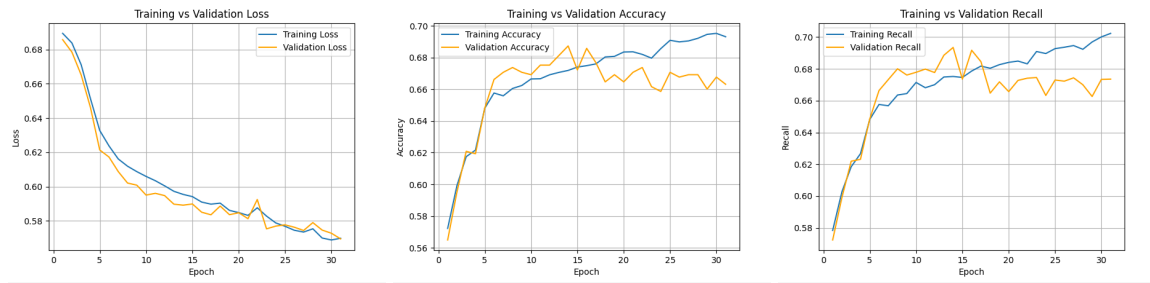


Figure 15: Results using bidirectional LSTM

Το LSTM, το οποίο χρησιμοποιούμε αποτελεί παραλλαγή ενός RNN (Recurrent Neural Network). Τα RNNs μπορούν να επεξεργάζονται αξιόπιστα εισόδους μεταβλητού μήκους και είναι αρκετά χρήσιμα στην επεξεργασία ακολουθιών δεδομένων, όπως στην περίπτωση αναγνώριση φυσικής γλώσσας. Το LSTM διαφέρει από το απλό RNN, δεν εφαρμόζει συναρτήσεις ενεργοποίησης στις εισόδους των αναδράσεων, εξασφαλίζοντας ότι οι ενημερώσεις θα είναι γραμμικές και κατ' επέκταση θα υπάρχει καλύτερη ροή πληροφορίας στο δίκτυο, καθώς τα σφάλματα δεν εξαφανίζονται από την επαναληπτική εφαρμογή των ενημερώσεων. Επιπλέον, χρησιμοποιεί μηχανισμό με θύρες που καθρίζουν πόσο επηρεάζεται κάθε διάνυσμα του δικτύου από τις αναδράσεις, καταφέροντας με αυτόν τον τρόπο αν αφομοιώνει μόνο τις σημαντικότερες πληροφορίες. Συνολικά, λοιπόν, η βελτίωση που πετυχαίνεται με χρήση LSTM είναι η δυνατότητα να χρησιμοποιούνται περισσότερες πληροφορίες για τις μακροχρόνιες σχέσεις των λέξεων.

Η χρήση ενός bidirectional LSTM οφείλει ώστε να μπορούμε να υπολογίσουμε για τις λέξεις αναπαραστάσεις οι οποίες δεν θα χρησιμοποιούν μόνο τα προηγούμενα συμφραζόμενα, αλλά και συμφραζόμενα από τις επόμενες λέξεις. Ουσιαστικά ένα bidirectional LSTM αποτελείται από δύο LSTM ένα που λειτουργεί όπως πριν και ένα που λειτουργεί αντίστροφα. Για την αναπαράσταση μπορούμε να συνενώσουμε τις αναπαραστάσεις για να έχουμε πιο ολοκληρωμένο αποτέλεσμα.

Έχουμε τα παρακάτω αποτελέσματα, στα οποία παρατηρούμε καλύτερα αποτελέσματα από τα αρχικά μοντέλα. Φυσικά, μεταξύ και των δύο αποτελεσμάτων που παρουσιάζονται, αν και δεν υπάρχουν σημαντικές διαφορές, το μοντέλο που χρησιμοποιεί bidirectional lstm είναι καλύτερο, αφού χρησιμοποιεί πιο πλήρη πληροφορία για τα συμφραζόμενα κάθε λέξης.

```
My train loss is : 0.5607251250554645
My test loss is : 0.5602150857448578
Accuracy for train: 0.701
Accuracy for test: 0.6691842900302115
F1 score for train: 0.6991579251732059
F1 score for test: 0.66653556075379
Recall score for train: 0.7049250586240421
Recall score for test: 0.6747360482654601
```

Figure 16: Metrics for simple LSTM (MR)

```
My train loss is : 0.5560546299767872
My test loss is : 0.5659464051326116
Accuracy for train: 0.71025
Accuracy for test: 0.6782477341389728
F1 score for train: 0.7091855463961967
F1 score for test: 0.6768844739284363
Recall score for train: 0.7125115601110581
Recall score for test: 0.6813075609031491
```

Figure 17: Metrics for bidirectional LSTM (MR)

5.3 Ερώτημα 3

Σε αυτό το ερώτημα θα χρησιμοποιήσουμε έναν απλό μηχανισμό attention για κατηγοριοποίηση συναισθήματος. Ένας τέτοιος μηχανισμός μπορεί να χρησιμοποιηθεί σε RNNs και LSTMs. Τα συγκεκριμένα δίκτυα δίνουν την τελευταία τους έξοδο ως την αναπαράσταση. Με τον μηχανισμό attention μπορούν να χρησιμοποιήθουν όλες οι καταστάσεις του δικτύου για να εξαχθεί η τελική αναπαράσταση, με κατάλληλα βάρη για κάθε έξοδο κατάστασης. Αυτό έχει ως αποτέλεσμα να μπορεί η αναπαράσταση να λαμβάνει υπόψη της και απομακρυσμένα σημεία της ακολουθίας λέξεων, τα οποία σημεία όμως περιέχουν σχετικές λέξεις.

Παρακάτω παρουσιάζεται ο κώδικας που έχουμε υλοποιήσει για τον μηχανισμό attention και με την χρήση average pooling για την εξαγωγή της τελικής αναπαράστασης και τα αντίστοιχα αποτελέσματα σε διαγράμματα και οι τιμές των μετρικών.

```
class Head(nn.Module):
    """ one head of self-attention """

    def __init__(self, head_size, n_embd, dropout=0.0):
        super().__init__()
        self.key = nn.Linear(n_embd, head_size, bias=False)
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False)

        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        B, T, C = x.shape
        k = self.key(x) # (B,T,C)
        q = self.query(x) # (B,T,C)
        # compute attention scores ("affinities")
```

```

        # (B, T, C) @ (B, C, T) -> (B, T, T)
        wei = q @ k.transpose(-2, -1) * C**-.5
        wei = F.softmax(wei, dim=-1) # (B, T, T)
        wei = self.dropout(wei)
        # perform the weighted aggregation of the values
        v = self.value(x) # (B,T,C)
        out = wei @ v # (B, T, T) @ (B, T, C) -> (B, T, C)
        return out

class FeedFoward(nn.Module):
    """ a simple linear layer followed by a non-linearity """

    def __init__(self, n_embd, dropout=0.0):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd),
            nn.ReLU(),
            nn.Linear(4 * n_embd, n_embd),
            nn.Dropout(dropout),
        )

    def forward(self, x):
        return self.net(x)

class SimpleSelfAttentionModel(nn.Module):

    def __init__(self, output_size, embeddings, max_length=60):
        super().__init__()

        self.n_head = 1
        self.max_length = max_length

        embeddings = np.array(embeddings)
        num_embeddings, dim = embeddings.shape

        self.token_embedding_table = nn.Embedding(num_embeddings, dim)
        self.token_embedding_table = self.token_embedding_table.from_pretrained(
            torch.Tensor(embeddings), freeze=True)
        self.position_embedding_table = nn.Embedding(self.max_length, dim)

        head_size = dim // self.n_head
        self.sa = Head(head_size, dim)
        self.ffwd = FeedFoward(dim)
        self.ln1 = nn.LayerNorm(dim)
        self.ln2 = nn.LayerNorm(dim)

        # TODO: Main-lab-Q3 - define output classification layer
        self.output = nn.Linear(dim, output_size)

    def forward(self, x, length=None):
        B, T = x.shape

```

```

device = x.device
tok_emb = self.token_embedding_table(x) # (B,T,C)
pos_emb = self.position_embedding_table(torch.arange(T, device=device))
# (T,C)
x = tok_emb + pos_emb # (B,T,C)
x = x + self.sa(self.ln1(x))
x = x + self.ffwd(self.ln2(x))

# TODO: Main-lab-Q3 - avg pooling to get a sentence embedding
x = x.mean(dim=1) # (B,C)

logits = self.output(x) # (C,output)
return logits

```

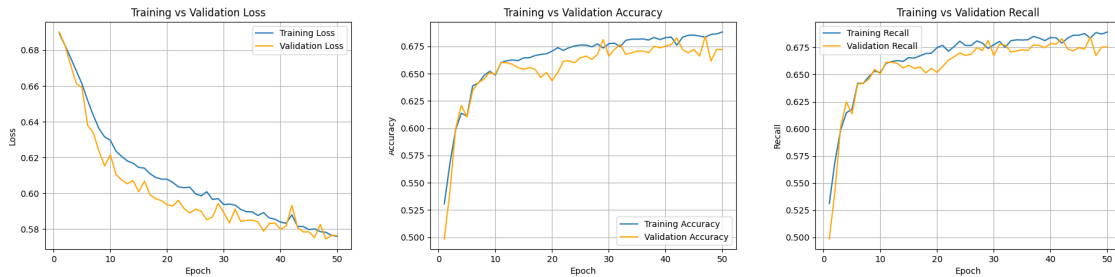


Figure 18: Results using attention mechanism

```

My train loss is : 0.5759195723230877
My test loss is : 0.5763992369174957
Accuracy for train: 0.68825
Accuracy for test: 0.6722054380664653
F1 score for train: 0.6876032606503614
F1 score for test: 0.6706837620552979
Recall score for train: 0.6892215472389289
Recall score for test: 0.6754482219907751

```

Figure 19: Metrics for model using attention mechanism (MR)

Συμβουλευόμαστε το tutorial “Let’s build GPT: from scratch, in code, spelled out” και καταλήγουμε στα παρακάτω συμπεράσματα για τα queries, queues και values της κλάσης attention.Head:

- **queries:** Στον μηχανισμό attention το query αποτελεί την “ερώτηση” που πραγματοποιείται για μια λέξη και εκφράζει πόση προσοχή θα πρέπει να δοθεί σε κάθε άλλη λέξη.
- **key:** Το key είναι η τιμή ή το χαρακτηριστικό του token με το οποίο θα συγκριθεί το query. Ουσιαστικά εκφράζει για κάθε λέξη πόσο σχετική είναι με την τρέχουσα λέξη.
- **value:** Το value περιέχει την τελική πληροφορία που χρειάζεται για το αποτέλεσμα. Μετά από τον υπολογισμό των attention scores από ζευγάρια queries - keys το σταθμισμένο άθροισμα των values δίνει το τελικό αποτέλεσμα της εξόδου.

Επιπλέον, για τα position_embeddings της κλάσης SimpleSelfAttentionModel καταλαβαίνουμε ότι κωδικοποιούν την χωρική πληροφορία για κάθε token. Ουσιαστικά, κρατάει την πληροφορία για την σειρά των tokens στο κείμενο. Αυτό είναι σημαντικό, καθώς ο μηχανισμός attention δεν διατηρεί πληροφορία για την σειρά των λέξεων και χρειάζεται με κάποιον τρόπο να διατηρείται η πληροφορία αυτή για να μπορεί να την χρησιμοποιήσει.

5.4 Ερώτημα 4

Επεκτείνουμε τον κώδικα του προηγούμενου ερωτήματος για να σχεδιάσουμε ένα μοντέλο με μηχανισμό attention πολλαπλών κεφαλών. Ο απλός μηχανισμός attention χρησιμοποιεί μόνο ένα set από queries, keys και values και με αυτόν τον τρόπο μπορεί να βρίσκει συσχετίσεις μεταξύ κομματιών του κειμένου, αλλά το κάνει δίνοντας βάση σε μία μόνο οπτική της ακολουθίας των δεδομένων. Το μοντέλο multi head attention χρησιμοποιεί πολλά heads κάθε ένα έχει το δικό του σύνολο από queries, keys και values. Με αυτόν τον τρόπο μπορεί να εξάγει από το κείμενο διαφορετικές οπτικές για την συσχέτιση των δεδομένων. Μπορεί για παράδειγμα το ένα head να δώσει βάση σε σημασιολογική σχέση μεταξύ σημείων του κειμένου, ένα άλλο στην συντακτική σχέση κλπ. Φυσικά, είναι πιο περίπλοκος μηχανισμός, όμως, μπορεί να εξάγει πιο ακριβείς και πλούσιες πληροφορίες από τα δεδομένα της εισόδου. Τα αποτελέσματα από όλα τα heads του μηχανισμού φυσικά συνεννώνονται για την παραγωγή του τελικού αποτελέσματος.

Παρακάτω παρουσιάζεται ο κώδικας για την υλοποίηση του μοντέλου και τα αντίστοιχα αποτελέσματα:

```
class MultiHeadAttention(nn.Module):
    """ multiple heads of self-attention in parallel """

    def __init__(self, num_heads, n_embd, dropout=0.0):
        super().__init__()
        self.head_size = n_embd // num_heads
        self.heads = nn.ModuleList([Head(self.head_size, n_embd)
                                     for _ in range(num_heads)])
        self.proj = nn.Linear(self.head_size * num_heads, n_embd)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        out = torch.cat([h(x) for h in self.heads], dim=-1)
        out = self.dropout(self.proj(out))
        return out

class MultiHeadAttentionModel(nn.Module):

    def __init__(self, output_size, embeddings, max_length=60, n_head=3):
        super().__init__()
```

```

self.n_head = n_head
self.max_length = max_length

embeddings = np.array(embeddings)
num_embeddings, dim = embeddings.shape

self.token_embedding_table = nn.Embedding(num_embeddings, dim)
self.token_embedding_table = self.token_embedding_table.from_pretrained(
    torch.Tensor(embeddings), freeze=True)
self.position_embedding_table = nn.Embedding(self.max_length, dim)

head_size = dim // self.n_head
self.msa = MultiHeadAttention(n_embd=dim, num_heads=self.n_head, dropout=0.1)
self.ffwd = FeedFoward(dim)
self.ln1 = nn.LayerNorm(dim)
self.ln2 = nn.LayerNorm(dim)

# TODO: Main-lab-Q3 - define output classification layer
self.output = nn.Linear(dim, output_size)

def forward(self, x, length=None):
    B, T = x.shape
    device = x.device
    tok_emb = self.token_embedding_table(x) # (B,T,C)
    pos_emb = self.position_embedding_table(torch.arange(T, device=device))
    # (T,C)
    x = tok_emb + pos_emb # (B,T,C)
    x = x + self.msa(self.ln1(x))
    x = x + self.ffwd(self.ln2(x))

    # TODO: Main-lab-Q3 - avg pooling to get a sentence embedding
    x = x.mean(dim=1) # (B,C)

    logits = self.output(x) # (C,output)
    return logits

```

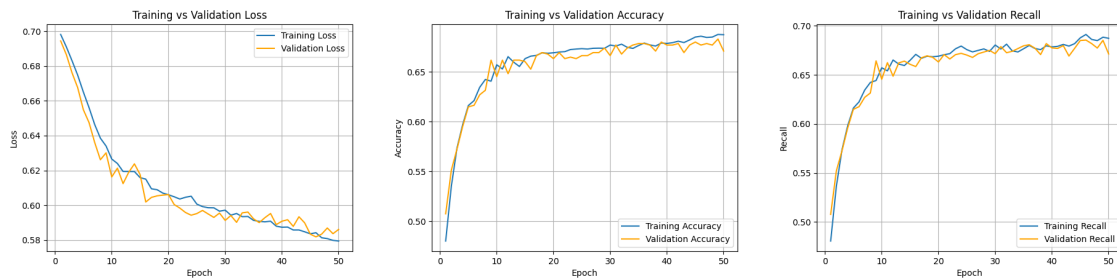


Figure 20: Results using multi-head attention mechanism

```
My train loss is : 0.5798121723863814
My test loss is : 0.5836324294408163
Accuracy for train: 0.687375
Accuracy for test: 0.6827794561933535
F1 score for train: 0.6866717690559212
F1 score for test: 0.6817307692307691
Recall score for train: 0.6884461633804044
Recall score for test: 0.6852206406023105
```

Figure 21: Metrics for model using multi head attention mechanism (MR)

Μπορούμε να παρατηρήσουμε μικρές διαφορές βελτίωσης σε σχέση με το προηγούμενο ερώτημα, οι οποίες όσο αυξάνουμε τις εποχές θα γίνονται πιο έντονες, αφού δεν έχει εφαρμοστεί early stopping μέχρι αυτό το σημείο.

5.5 Ερώτημα 5

Στην συνέχεια, θα χρησιμοποιήσουμε έναν Transformer - Encoder για την κατηγοριοποίηση συναισθήματος. Στην αρχιτεκτονική των transformers δεν είναι απαραίτητη η χρήση συνελικτικών δικτύων ή αναδρομικών δικτύων, αλλά μόνο μηχανισμών προσοχής και κάποια άλλα στοιχεία, όπως στρώματα κανονικοποίησης για παράδειγμα. Το σύνολο των επιπέδων του transformer encoder περιλαμβάνει έναν μηχανισμό multihead attention, ένα μικρό feedforward network και ένα στρώμα κανονικοποίησης. Τα επίπεδα αυτά τοποθετούνται στην σειρά, σχηματίζοντας τον τελικό transformer encoder. Τελικά, ένας transformer-encoder δίνει την μέγιστη έμφαση στην συσχέτιση των λέξεων. Σημειώνεται και εδώ ότι επειδή ο μηχανισμός attention δεν κωδικοποιεί χωρική πληροφορία για τα δεδομένα του κειμένου, αυτό πραγματοποιείται όπως και στα προηγούμενα ερωτήματα.

Ο κώδικας για το μοντέλο που υλοποιήσαμε φαίνεται παρακάτω. Για τις διαφορές του με το μοντέλο του προηγούμενου ερωτήματος μπορούμε να δούμε ότι το προηγούμενο μοντέλο χρησιμοποιεί μόνο ένα επίπεδο multi head attention, ενώ ο transformer - encoder χρησιμοποιεί ένα block το οποίο περιλαμβάνει τον μηχανισμό multihead attention ως βασική δομική μονάδα του, αντίγραφα της οποίας τοποθετεί σε σειρά για να δημιουργήσει την τελική του αρχιτεκτονική. Μπορούμε, τέλος, να σχολιάσουμε ότι το μοντέλο του προηγούμενου ερωτήματος στην πραγματικότητα μοιάζει με έναν transformer - encoder που χρησιμοποιεί ένα μόνο block.

```
class Block(nn.Module):
    """ Transformer block: communication followed by computation """

    def __init__(self, n_head, n_embd, dropout=0.1):
        super().__init__()
        self.sa = MultiHeadAttention(num_heads=n_head, n_embd=n_embd, dropout=dropout)
        self.ffwd = FeedForward(n_embd)
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)

    def forward(self, x):
        x = x + self.sa(self.ln1(x))
        x = x + self.ffwd(self.ln2(x))
        return x

class TransformerEncoderModel(nn.Module):
    def __init__(self, output_size, embeddings, max_length=60, n_head=3, n_layer=3):
        super().__init__()

        self.n_head = n_head
```

```

self.max_length = max_length

embeddings = np.array(embeddings)
num_embeddings, dim = embeddings.shape
self.dim = dim

self.token_embedding_table = nn.Embedding(num_embeddings, dim)
self.token_embedding_table = self.token_embedding_table.from_pretrained(
    torch.Tensor(embeddings), freeze=True)
self.position_embedding_table = nn.Embedding(self.max_length, dim)

self.blocks = nn.Sequential(
    *[Block(n_head=n_head, n_embd=dim, dropout=0.1) for _ in range(n_layer)]
)
self.ln_f = nn.LayerNorm(dim)

self.output = nn.Linear(dim, output_size)

def forward(self, x, length=None):
    B, T = x.shape
    device = x.device
    tok_emb = self.token_embedding_table(x) # (B,T,C)
    pos_emb = self.position_embedding_table(torch.arange(T, device=device))
    # (T,C)
    x = tok_emb + pos_emb # (B,T,C)

    x = self.blocks(x)
    x = self.ln_f(x)
    x = x.mean(dim=1) # avg pooling

    logits = self.output(x)
    return logits

```

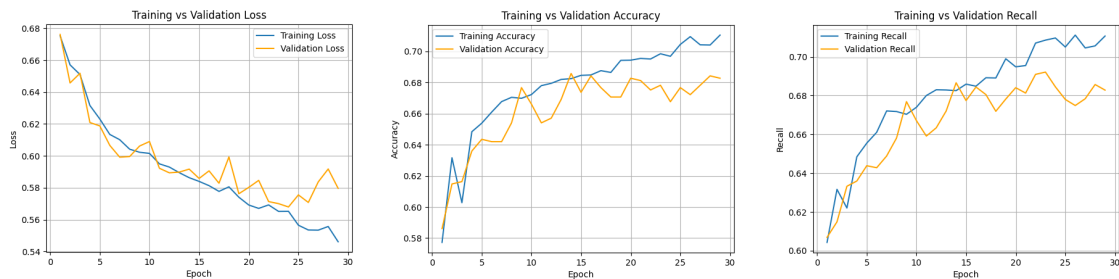


Figure 22: Results using transformer - encoder

```

My train loss is : 0.5798121723863814
My test loss is : 0.5836324294408163
Accuracy for train: 0.687375
Accuracy for test: 0.6827794561933535
F1 score for train: 0.6866717690559212
F1 score for test: 0.6817307692307691
Recall score for train: 0.6884461633804044
Recall score for test: 0.6852206406023105

```

Figure 23: Metrics for model using transformer - encoder (MR)

Πάνω φαίνονται τα αποτελέσματα για τον transformer - encoder, ενδεικτικά, πριν πειραματιστούμε με τις παραμέτρους του για αριθμό layers ίσο με 3 και αριθμό heads σε κάθε layer ίσο με 3. Στην συνέχεια πειραματιζόμαστε αρχικά με τον αριθμό των layers. Κρατάμε σταθερό τον αριθμό heads για κάθε layer και παρυσιάζουμε τις μετρικές του για αριθμό επιπέδων 6, 9 και 12. Στην συνέχεια, για αριθμό επιπέδων ίσο με 3 και ίσο με 6 δοκιμάζουμε αριθμό heads 6 και 9.

<pre> My train loss is : 0.5406446442717597 My test loss is : 0.5637033035357794 Accuracy for train: 0.7185 Accuracy for test: 0.6842900302114804 F1 score for train: 0.7184819828469022 F1 score for test: 0.6839720148103143 Recall score for train: 0.7184819828469022 Recall score for test: 0.6850348240469208 </pre>	<pre> My train loss is : 0.48958279925679404 My test loss is : 0.57057686150074 Accuracy for train: 0.7615 Accuracy for test: 0.675226586102719 F1 score for train: 0.7612851566409768 F1 score for test: 0.67431621966093 Recall score for train: 0.7620004492146766 Recall score for test: 0.6772079456505686 </pre>	<pre> My train loss is : 0.5579416207850926 My test loss is : 0.5876692732175192 Accuracy for train: 0.704375 Accuracy for test: 0.6737160120845922 F1 score for train: 0.7042981128890091 F1 score for test: 0.6736087651221183 Recall score for train: 0.70439819891585 Recall score for test: 0.6739446330320371 </pre>
--	--	--

Figure 24: Number of layers: 6, 9 and 12 (left to right) - number of heads: 3

<pre> My train loss is : 0.5461290483436887 My test loss is : 0.5703651408354441 Accuracy for train: 0.71275 Accuracy for test: 0.6812688021752266 F1 score for train: 0.7126542959214353 F1 score for test: 0.6811459225018548 Recall score for train: 0.7128088539916547 Recall score for test: 0.6815489249670907 </pre>	<pre> My train loss is : 0.5430454758424608 My test loss is : 0.5709549685319265 Accuracy for train: 0.713125 Accuracy for test: 0.6873111782477341 F1 score for train: 0.7123805645020187 F1 score for test: 0.6862221591260365 Recall score for train: 0.7146364532019704 Recall score for test: 0.6899481673454275 </pre>
---	--

Figure 25: Number of layers: 6 - number of heads: 6 and 9

<pre> My train loss is : 0.5811147311377147 My test loss is : 0.5811512668927511 Accuracy for train: 0.687625 Accuracy for test: 0.676737160120846 F1 score for train: 0.6874578632088801 F1 score for test: 0.6765482159328566 Recall score for train: 0.6877466851345196 Recall score for test: 0.677151090983944 </pre>	<pre> My train loss is : 0.5530802361548893 My test loss is : 0.5723056842883428 Accuracy for train: 0.70825 Accuracy for test: 0.6858006042296072 F1 score for train: 0.7081631603003868 F1 score for test: 0.6856600189933524 Recall score for train: 0.7082902718434021 Recall score for test: 0.6861335893567412 </pre>
--	---

Figure 26: Number of layers: 3 - number of heads: 6 and 9

Κατα την διάρκεια της εκπαίδευσης μπορούμε σε 1η φάση να παρατηρήσουμε ότι ο χρόνος εκπαίδευσης για μεγαλύτερο αριθμό επιπέδων ή μεγαλύτερο αριθμό heads κάθε layer αυξάνεται σημαντικά, κάτι το οποίο φαίνεται λογικό, αφού οι παράμετροι αυξάνονται.

Κρατώντας σταθερό τον αριθμό των heads ανά layer και αυξάνοντας τον αριθμό των layers μπορούμε να παρατηρήσουμε ότι η ακρίβεια στο training set αυξάνεται μέχρι αριθμό layers ίσο με 9 και στην συνέχεια μειώνεται. Αντίστοιχα, στα ίδια διαστήματα βελτιώνονται και οι υπόλοιπες μετρικές, δηλαδή αυξάνονται το f1 score και το recall και μειώνεται το loss στο training set (μέχρι αριθμό layers ίσο με 9). Παρατηρούμε ότι στο validation set υπάρχει κάποια μικρή βελτίωση στις μετρικές, η οποία όμως ειδικά σε αρκετά υψηλές τιμές φαίνεται να αντιστρέφεται. Μπορούμε να συμπεράνουμε για αυτό, ότι αυξάνοντας τον αριθμό των επιπέδων μέχρι ένα σημείο το μοντέλο έχει τις απαραίτητες παραμέτρους για την αναγνώριση και τον διαχωρισμό σύνθετων χαρακτηριστικών στα κείμενα. Όμως, επειδή το dataset που χρησιμοποιούμε είναι σχετικά μικρό, σε μεγαλύτερο αριθμό επιπέδων έχουμε overfitting. Φυσικά, το τελευταίο συμπέρασμα ίσως ισχύει για διαφορετικό αριθμό παραμέτρων σε κάποιο μεγαλύτερο dataset.

Για σταθερό αριθμό layers και αυξάνοντας τον αριθμό των heads ανά layer παρατηρούμε μικρή βελτίωση των μετρικών. Αυτό φαίνεται λογικό, καθώς, όταν αυξάνουμε τον αριθμό των heads ανά επίπεδο, αναλύονται περισσότερα χαρακτηριστικά του κειμένου σε κάθε επίπεδο και το μοντέλο γίνεται πιο αξιόπιστο για μεγαλύτερα και περίπλοκα κείμενα. Σε αυτήν την περίπτωση, όπως και στην περίπτωση που αυξάνουμε τον αριθμό των layers σε κάποιο μεγαλύτερο dataset θα ήταν ακόμα πιο σημαντικό να αυξήσουμε τον αριθμό των heads, ώστε να μπορεί το μοντέλο να λειτουργεί με αξιοπιστία, με προσοχή όμως ότι και σε αυτήν την περίπτωση θα μπορούσε αν αυξήσουμε τον αριθμό των heads πολύ να οδηγηθούμε σε overfitting ή το μοντέλο να αναλύει και χαρακτηριστικά του κειμένου τα οποία δεν είναι τόσο σημαντικά και να δίνει βαρύτητα σε θόρυβο πρακτικά για τις προβλέψεις του.

5.6 Ερώτημα 6

Θα χρησιμοποιήσουμε προεκπαιδευμένα μοντέλα transformers για την κατηγοριοποίηση συναισθήματος. Επιλέγουμε 3 μοντέλα για κάθε dataset από το <https://huggingface.co/models>. Παρακάτω φαίνεται ο κώδικας μας που πραγματοποιεί τις προβλέψεις για τα μοντέλα αυτά και τα αποτελέσματα σε μορφή πίνακα.

```
from transformers import pipeline
from sklearn.preprocessing import LabelEncoder
from tqdm import tqdm
from utils.load_datasets import load_MR, load_Semeval2017A
from training import get_metrics_report
import torch

# Label mappings for each dataset and model
LABELS_MAPPING = {
    'MR': {
        'siebert/sentiment-roberta-large-english': {
            'POSITIVE': 'positive', # Positive sentiment
            'NEGATIVE': 'negative', # Negative sentiment
        },
        'distilbert-base-uncased-finetuned-sst-2-english': {
            'POSITIVE': 'positive', # Positive sentiment
            'NEGATIVE': 'negative', # Negative sentiment
        },
        'textattack/bert-base-uncased-SST-2': {
            'LABEL_0': 'negative', # Negative sentiment
            'LABEL_1': 'positive', # Positive sentiment
        }
    },
    'Semeval2017A': {
        'finiteautomata/bertweet-base-sentiment-analysis': {
            'NEG': 'negative', # Negative sentiment
            'NEU': 'neutral', # Neutral sentiment
            'POS': 'positive', # Positive sentiment
        },
        'cardiffnlp/twitter-roberta-base-sentiment': {
            'LABEL_0': 'negative', # Negative sentiment
            'LABEL_1': 'neutral', # Neutral sentiment
            'LABEL_2': 'positive', # Positive sentiment
        },
        'j-hartmann/sentiment-roberta-large-english-3-classes': {
            'negative': 'negative',
            'neutral': 'neutral',
```

```

        'positive': 'positive',
    }
}
}

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# Datasets
DATASETS = ['MR', 'Semeval2017A']
MODEL_NAMES = {
    'MR': ['siebert/sentiment-roberta-large-english',
           'distilbert-base-uncased-finetuned-sst-2-english',
           'textattack/bert-base-uncased-SST-2'],
    'Semeval2017A': ['finiteautomata/bertweet-base-sentiment-analysis',
                     'cardiffnlp/twitter-roberta-base-sentiment',
                     'j-hartmann/sentiment-roberta-large-english-3-classes']
}

if __name__ == '__main__':
    # Loop through each dataset
    for dataset in DATASETS:
        # Load the appropriate dataset
        if dataset == "Semeval2017A":
            X_train, y_train, X_test, y_test = load_Semeval2017A()
        elif dataset == "MR":
            X_train, y_train, X_test, y_test = load_MR()
        else:
            raise ValueError("Invalid dataset")

        # Encode labels
        le = LabelEncoder()
        le.fit(list(set(y_train))) # Fit label encoder to the training labels
        y_train = le.transform(y_train)
        y_test = le.transform(y_test)
        n_classes = len(list(le.classes_))

        # Loop through each model
        for model_name in MODEL_NAMES[dataset]:
            print(f"Evaluating model: {model_name}")

            # Define the sentiment-analysis pipeline for the model
            sentiment_pipeline = pipeline("sentiment-analysis",
                                           model=model_name, device=DEVICE)

            y_pred = []

            # Using tqdm for progress bar that updates in place
            for x in tqdm(X_test, desc=f"Processing {model_name}", leave=True):
                # The 'desc' is the description for the progress bar
                # Get the label using the defined pipeline
                result = sentiment_pipeline(x)[0]
                label = result['label']

```

```

y_pred.append(LABELS_MAPPING[dataset][model_name][label])

# Ensure the predictions are transformed using the same encoder
y_pred_enc = le.transform(y_pred) #Transform predictions using the encoder

# Print evaluation metrics
print(f'\nDataset: {dataset}\nPre-Trained model: {model_name}\nTest
set evaluation\n{get_metrics_report([y_test], [y_pred_enc])}')

```

Επισημαίνουμε ότι έχουμε επιλέξει μοντέλα που έχουν labels "positive" και "negative" για το dataset MR, ενώ για το Semeval επιλέγουμε μοντέλα που κατηγοριοποιούν και ουδέτερο συναίσθημα. Με αυτόν τον τρόπο μπορούμε να έχουμε πιο ολοκληρωμένο μέτρο σύγκρισης μεταξύ των μοντέλων. Φυσικά, επειδή μπορεί τα μοντέλα να μην έχουν τα ίδια labels με τα δεδομένα του dataset φροντίζουμε να πραγματοποιήσουμε το κατάλληλο label mapping.

Table 3: Performance of models on MR and Semeval2017A datasets

Dataset	Model	Accuracy	Recall	F1 Score
MR	siebert/sentiment-roberta-large-english	0.92598	0.92598	0.92598
	distilbert-base-uncased-finetuned-sst-2-english	0.89123	0.89123	0.89121
	textattack/bert-base-uncased-SST-2	0.89879	0.89879	0.89873
Semeval2017A	finiteautomata/bertweet-base-sentiment-analysis	0.71776	0.73018	0.71805
	cardiffnlp/twitter-roberta-base-sentiment	0.72378	0.72294	0.72221
	j-hartmann/sentiment-roberta-large-english-3-classes	0.69846	0.64874	0.66936

Το πρώτο που μπορούμε να παρατηρήσουμε για τα πάνω μοντέλα είναι ότι στα περισσότερα από αυτά η τιμή του accuracy είναι σχεδόν ίδια με την τιμή του recall (έχουν πολύ μικρή διαφορά αν παρουσιάσουμε περισσότερα δεκαδικά ψηφία). Αυτό δεν μας φαίνεται περίεργο σε συνδυασμό με το δεδομένο ότι το γεγονός αυτό συνοδεύεται από πολύ υψηλές τιμές των μετρικών, κάτι το οποίο σημαίνει ότι το μοντέλο αυτό είναι πολύ αξιόπιστο στις προβλέψεις. Με δεδομένα αυτά υποθέτουμε ότι τα μοντέλα που εμφανίζεται το φαινόμενο αυτό κάνουν λίγες false positive και false negative προβλέψεις. Αυτό οδηγεί σε πολύ παρόμοιες τιμές accuracy και recall. Φυσικά, αντίστοιχα προκύπτει και το συμπέρασμα γιατί το f1 score έχει μικρή διαφορά με τις υπόλοιπες μετρικές.

Για τα μοντέλα που χρησιμοποιούμε στο MR dataset παρατηρούμε ότι το καλύτερο είναι το siebert/sentiment-roberta-large-english με πολύ υψηλή απόδοση και ακολουθούν τα textattack/bert-base-uncased-SST-2 και distilbert-base-uncased-finetuned-sst-2-english με σειρά απόδοσης, τα οποία είναι βασισμένα σε παρόμοια αρχιτεκτονική και έχουν βελτιστοποιηθεί με την χρήση του ίδιου dataset.

Για τα μοντέλα που χρησιμοποιούμε στο Semeval2017A dataset αρχικά μπορούμε να παρατηρήσουμε ότι συγκριτικά με τα μοντέλα που χρησιμοποιούμε στο MR dataset έχουν χειρότερη απόδοση. Υποθέτουμε ότι αυτό συμβαίνει επειδή το dataset αυτό είναι μεγαλύτερο και περισσότερο πολύπλοκο από το MR. Μεταξύ των τριών αυτών μοντέλων παρατηρούμε ότι το καλύτερο μοντέλο είναι το cardiffnlp/twitter-roberta-base-sentiment και ακολουθούν τα finiteautomata/bertweet-base-sentiment-analysis και j-hartmann/sentiment-roberta-large-english-3-classes με σειρά απόδοσης.

Τέλος, μπορούμε να σημειώσουμε ότι τα μοντέλα που έχουν τα καλύτερα αποτελέσματα και για τα δύο dataset έχουν αρχιτεκτονική με transformers-encoders και είναι και τα δύο βασισμένα στην βελτιστοποίηση του BERT. Φυσικά, έχουν διαφορετικές χρήσεις, έχουν εκπαιδευτεί με διαφορετικό τρόπο και αναγνωρίζουν διαφορετικές κλάσεις, καθώς το ένα αναγνωρίζει και ουδέτερο συναίσθημα.

5.7 Ερώτημα 7

Στην συνέχεια θα κάνουμε fine-tune pre-trained transformer models. Αρχικά, εκτελούμε τον κώδικα του αρχείου `finetune_pretrained.py` όπως μας έχει δοθεί και έχουμε τα παρακάτω αποτελέσματα:

```
{ 'eval_loss': 0.3885692659993744, 'eval_accuracy': 0.9, 'eval_runtime': 0.288, 'eval_samples_per_second': 138.887, 'eval_steps_per_second': 17.361, 'epoch': 1.0}
{ 'eval_loss': 0.28198426961898804, 'eval_accuracy': 0.9, 'eval_runtime': 0.2877, 'eval_samples_per_second': 139.853, 'eval_steps_per_second': 17.382, 'epoch': 2.0}
{ 'eval_loss': 0.31483882665634155, 'eval_accuracy': 0.9, 'eval_runtime': 0.2822, 'eval_samples_per_second': 141.733, 'eval_steps_per_second': 17.717, 'epoch': 3.0}
{ 'eval_loss': 0.3237629532814026, 'eval_accuracy': 0.9, 'eval_runtime': 0.2938, 'eval_samples_per_second': 136.154, 'eval_steps_per_second': 17.019, 'epoch': 4.0}
{ 'eval_loss': 0.32668335382352905, 'eval_accuracy': 0.9, 'eval_runtime': 0.268, 'eval_samples_per_second': 149.256, 'eval_steps_per_second': 18.657, 'epoch': 5.0}
{ 'train_runtime': 6.9772, 'train_samples_per_second': 28.665, 'train_steps_per_second': 3.583, 'train_loss': 0.19259111404418947, 'epoch': 5.0}
```

Figure 27: Fine tuning

Στην συνέχεια δοκιμάζουμε να κάνουμε fine tuning σε ολόκληρο το dataset με χρήση gpu. Σημειώνουμε ότι επειδή τα μοντέλα έχουν πολλές παραμέτρους, όπως και οι αντίστοιχοι trainers και τα dataset χρησιμοποιούμε 5 εποχές και batch size ίσο με 8, ώστε να μπορούμε να παράξουμε αποτελέσματα με το διαθέσιμο hardware.

Table 4: Performance of fine tuned models on MR and Semeval2017A datasets

Dataset	Model	Accuracy
MR	siebert/sentiment-roberta-large-english	0.92376
	distilbert-base-uncased-finetuned-sst-2-english	0.889728
	textattack/bert-base-uncased-SST-2	0.877644
Semeval2017A	finiteautomata/bertweet-base-sentiment-analysis	0.692934
	cardiffnlp/twitter-roberta-base-sentiment	0.688701
	j-hartmann/sentiment-roberta-large-english-3-classes	0.66642

Παρατηρούμε για τα αποτελέσματα που έχουν προκύψει ότι εμφανίζουν μια μείωση από αυτά των μοντέλων πριν το fine tuning. Αν και η αρχική μας σκέψη ήταν ότι πιθανόν να οφείλεται σε overfitting αυτό που παρατηρούμε, με πιο ενδελεχή έρευνα και στοχευμένη σκέψη καταλήγουμε ότι πιθανότατα οφείλεται στον μικρό αριθμό εποχών. Επειδή ο αριθμός εποχών είναι μικρός και πιθανολογούμε ότι αντίστοιχα η παράμετρος learning rate είναι μικρή, τα μοντέλα στην πραγματικότητα σε 5 εποχές δεν μαθαίνουν κάτι από τα δεδομένα του fine tuning. Αντίθετα, μάλιστα, επειδή τα μοντέλα αυτά είναι βελτιστοποιημένα, ακόμα αν και αυτή η βελτιστοποίηση είναι σε άλλο παρόμοιο dataset, η προσπάθεια για εκμάθηση νέων στοιχείων μεταβάλει λίγο τις παραμέτρους και για αυτό παρατηρούμε πτώση στην ακρίβεια, η οποία είναι σε μια τάξη κάτω από 1% στα περισσότερα μοντέλα. Υποθέτουμε, λοιπόν, ότι για fine tuning αρκετά μεγάλο αριθμό εποχών, ο οποίος φυσικά να μην οδηγεί σε overfitting θα παρατηρούσαμε κάποια σημαντική βελτίωση στην απόδοση του μοντέλου.

Επειδή όπως παρατηρούμε στο fine tuning με ένα μέρος του dataset έχουμε αρκετά παρόμοια αποτελέσματα με όταν χρησιμοποιούμε το σύνολο του dataset θα πειραματιστούμε, ελέγχοντας αν σε μικρό ποσοστό του dataset, αλλά μεγάλο αριθμό εποχών μπορούμε να δούμε βελτίωση, ώστε να γλιτώσουμε την υπολογιστικά βαριά και χρονοβόρα διαδικασία fine tuning με χρήση όλου του dataset.

Συγκεκριμένα θα πειραματιστούμε με το μοντέλο "distilbert-base-uncased-finetuned-sst-2-english" για μικρότερο μέγεθος του dataset στο training και ολόκληρο το evaluation set και παρουσιάζουμε τα αποτελέσματα της ακρίβειας στο evaluation set:

Table 5: Performance of distilbert-base-uncased-finetuned-sst-2-english for different parameter values and 10% of the training set

epochs number	batch size	Accuracy
25	8	0.901813
40	16	0.903323
40	8	0.897281
25	16	0.89577

Παρατηρούμε ότι για μικρότερο κομμάτι του training set μπορούμε να ανεβάσουμε σημαντικά τον αριθμό των εποχών και να πειραματιστούμε με το μέγεθος του batch σε σημαντικά λιγότερο χρόνο. Παρατηρούμε μάλιστα ότι μπορούμε να πραγματοποιήσουμε fine tuning σε ένα μοντέλο με βελτίωση στην επίδοση σε ολόκληρο το evaluation set, χρησιμοποιώντας μόνο ένα κλάσμα του training set σε αυτή την διαδικασία. Συγκεκριμένα, έχουμε χρησιμοποιήσει το 10% του training set και όλο το evaluation set, για να είναι τα συμπεράσματα μας ρεαλιστικά για την τελική απόδοση του μοντέλου.

Μπορούμε να παρατηρήσουμε από τα μοντέλα που έχουν προκύψει πάνω ότι κάποια είναι αρκετά αξιόπιστα. Ωστόσο, αν θέλουμε να πετύχουμε ακόμα καλύτερα αποτελέσματα, καταναλώνοντας λιγότερους πόρους και χρόνο μπορούμε να ακολουθήσουμε την διαδικασία που πραγματοποιούμε πάνω για έρεση των βέλτιστων υπερπαραμέτρων ή έστω μια αρκετά καλή εκτίμηση τους και στην συνέχεια να προχωρήσουμε σε fine tuning με όλο το test set και ελάχιστες αλλαγές των βέλτιστων παραμέτρων. Με αυτόν τον τρόπο θα μπορούμε να βρούμε ταχύτερα τις βέλτιστες παραμέτρους και με βάση αυτές να παράξουμε το καλύτερο δυνατό μοντέλο.

5.8 Ερώτημα 8

Συνδεόμαστε στο ChatGPT και ζητάμε να μας εξηγήσει τον κώδικα του tutorial "Let's build GPT: from scratch, in code, spelled out".

Αρχικά, το ChatGPT εξήγησε τα πρώτα κελιά του jupyter notebook, το οποίο δώσαμε ως αρχείο και ρώτησε αν θέλουμε να συνεχίσει την ανάλυση κελί - κελί ή να μας δώσει μια γενική ιδέα - ανάλυση του κώδικα. Ζητήσαμε πρώτα να πραγματοποιήσει την ανάλυση κελί - κελί και στην συνέχεια να δώσει μια γενική ιδέα του κώδικα. Παρατηρούμε ότι η ανάλυση του κώδικα κελί - κελί μοιάζει όπως παρακάτω για κάθε κελί ή ομάδα κελιών και στο τέλος υπάρχει ένα γενικό συμπέρασμα για τον κώδικα.

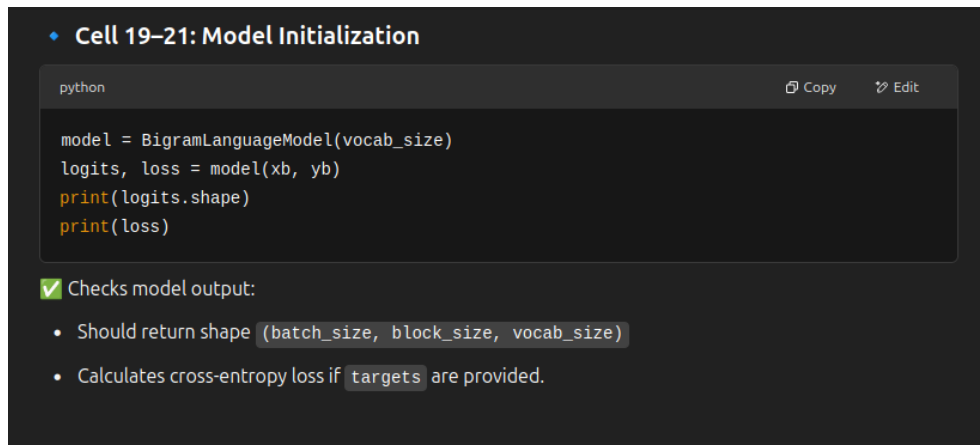


Figure 28: ChatGPT cell by cell explanation

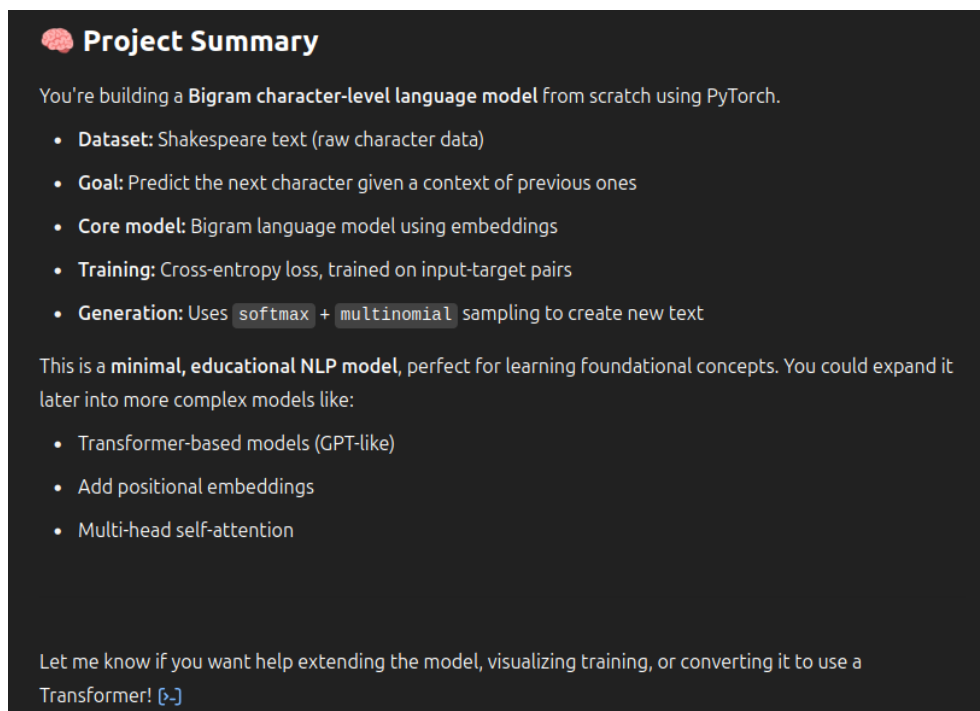


Figure 29: ChatGPT summary

Μπορούμε να παρατηρήσουμε, αφού πρώτα κατανοήσουμε τον κώδικα χωρίς την βοήθεια του ότι δίνει αρκετά αξιόπιστη περιγραφή της λειτουργικότητας του κώδικα. Στην συνέχεια ζητάμε από το Chat να αξιολογήσει τον κώδικα και χωρίζει σε κατηγορίες την αξιολόγηση του:

- Functionality / Correctness
- Code Quality / Readability
- Model Design
- Extensibility
- Performance Monitoring
- Overall Evaluation

Στις περισσότερες κατηγορίες βρίσκει σωστό τον κώδικα και επισημαίνει κάποια σημεία τα οποία θεωρεί ότι θα μπορούσαν να βελτιωθούν.

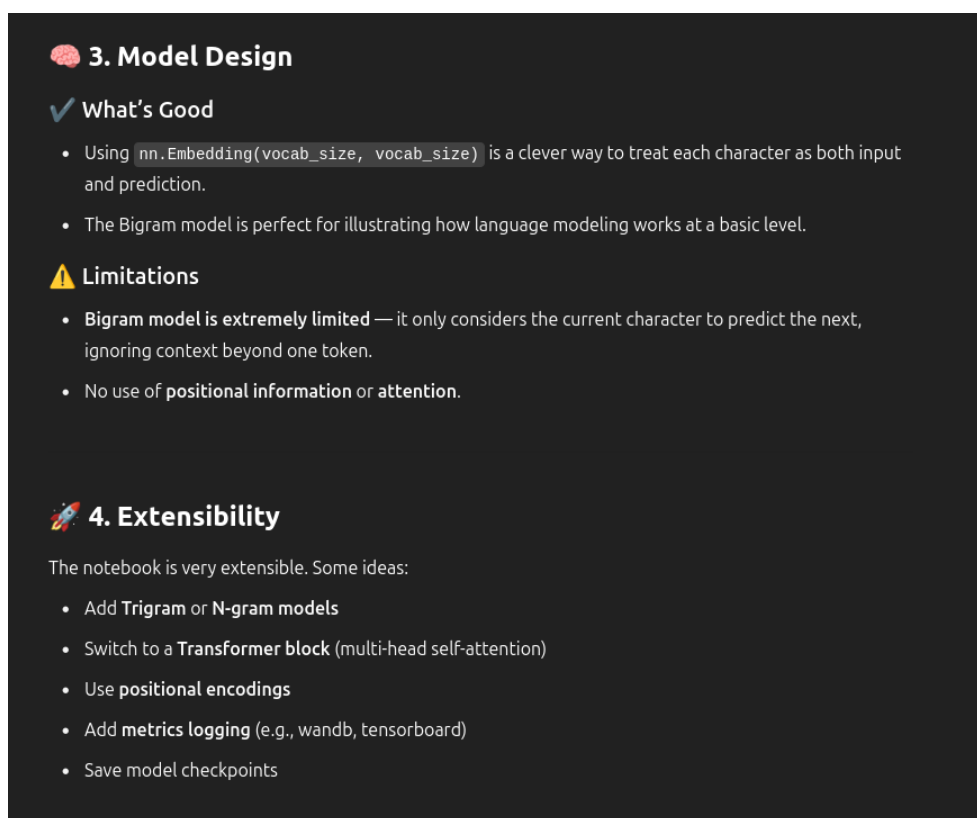


Figure 30: ChatGPT code review

Overall Evaluation	
Category	Evaluation
Functionality	✓ Works as intended
Code Style	✓ Clear, well-commented
Model Simplicity	✓ Excellent for education
Scalability	⚠ Limited but extendable
Best Practices	⚠ Missing monitoring/validation

Figure 31: ChatGPT overall code evaluation

Αφού έχουμε ζητήσει αξιολόγηση του κώδικα, ζητάμε σε συνέχεια της προηγούμενης ερώτησης να πραγματοποιήσει βελτιστοποίηση του κώδικα που έχουμε δώσει. Παρατηρούμε από τις παρακάτω φωτογραφίες και από την συνολική μας εμπειρία ότι οι περισσότερες βελτιώσεις που προτείνει αφορούν την ωραιοποίηση του κώδικα, όπως ομαδοποίηση συναρτήσεων σε κλάσεις, των αρχείων κώδικα ή και των μεταβλητών απλά σε συναρτήσεις configuration. Αυτό μπορεί να βοηθήσει σημαντικά στην αναγνωσιμότητα του κώδικα και την κατανόηση του κατ' επέκταση και φυσικά την ικανότητα μας να τον διαχειριζόμαστε καλύτερα και πιο γρήγορα. Ωστόσο, οι αλλαγές αυτές δεν βελτιώνουν σημαντικά την ουσιαστική επίδοση του κώδικά μας στο κομμάτι το οποίο καλούμαστε να πραγματοποιήσουμε την βελτιστοποίηση. Παρατηρεί, φυσικά, όπως φαίνεται και παρακάτω ότι το μοντέλο που χρησιμοποιούμε μπορεί να βελτιωθεί και πραγματοποιεί προτάσεις για καλύτερο μοντέλο για βελτιστοποίηση. Από την εμπειρία μας στην εργαστηριακή άσκηση κρίνουμε ότι η παρατήρηση του αυτή είναι εύστοχη.

Συνολικά, κρίνουμε ότι από την συνομιλία με το ChatGPT μπορεί να μειωθεί ο απαραίτητος χρόνος για να κατανοήσουμε έναν κώδικα. Επιπλέον, μπορούμε να βελτιώσουμε τον κώδικα μας σε θέματα εικόνας και αναγνωσιμότητας, να παρουσιάζουμε καλύτερα τα αποτελέσματά μας, να έχουμε μια πιο στοχευμένη εικόνα στο documentation βιβλιοθηκών και να επιλύουμε με σχετική ευκολία μικρά σφάλματα στον κώδικα μας. Με αυτόν τον τρόπο μπορούμε να γλιτώσουμε σημαντικό χρόνο κατά την βελτιστοποίηση του κώδικα μας. Στο πρακτικό κομμάτι του κώδικα, μπορούμε να παρατηρήσουμε ότι το Chat κάνει εύστοχη παρατήρηση για την βελτιστοποίηση του κώδικα, ωστόσο για πραγματικά αποτελέσματα βελτιστοποίησης σε θέματα έρευνας ή τεχνολογίας αιχμής, για τα οποία υπάρχουν σχετικά ελάχιστα δεδομένα στον χώρο του διαδικτύου και κατ' επέκταση πιθανολογούμε στα δεδομένα εκπαίδευσης του Chat θεωρούμε ότι σε αυτές τις περιπτώσεις δεν μπορεί να είναι τόσο αξιόπιστο.

Παρακάτω φαίνονται οι προτάσεις για αλλαγές στον κώδικα, αλλά και ο κώδικας για ένα μοντέλο transformer - encoder που ζητήθηκε σε συνέχεια της πρότασης για βελτίωση.

1. Core Code Improvements

✓ Refactor Configs & Hyperparameters

Move all constants (like `block_size`, `batch_size`, `vocab_size`, `learning_rate`, etc.) into a config dictionary or a simple class:

```
python
class Config:
    block_size = 8
    batch_size = 64
    vocab_size = len(chars)
    learning_rate = 1e-3
    device = 'cuda' if torch.cuda.is_available() else 'cpu'
```

Then reference `Config.block_size` etc., for clarity and easy tuning.

Figure 32: ChatGPT improvements

Summary: What I'd Change

Area	Change
Code Structure	Modularize tokenizer, data loading, configs
Model	Replace Bigram with Transformer or LSTM
Training	Track val loss, use GPU, log progress, plot curves
Scaling	Move from notebook to Python package structure
Usability	Add checkpoints, generation UI, sampling temperature

If you'd like, I can help you **refactor your current notebook into a production-ready script or modular project structure** — just let me know your target goals (e.g., "transformer version" or "serve as API" or "training on a new dataset").

Figure 33: ChatGPT summary of changes

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class TransformerLanguageModel(nn.Module):
    def __init__(self, vocab_size, n_embd=128, n_head=4,
                 n_layer=4, block_size=128, dropout=0.1):
        super().__init__()
        self.token_embedding = nn.Embedding(vocab_size, n_embd)
        self.position_embedding = nn.Embedding(block_size, n_embd)
```

```

transformer_layer = nn.TransformerEncoderLayer(
    d_model=n_embd,
    nhead=n_head,
    dim_feedforward=4 * n_embd,
    dropout=dropout,
    activation='gelu',
    batch_first=True
)
self.transformer = nn.TransformerEncoder(transformer_layer, num_layers=n_layer)
self.ln_f = nn.LayerNorm(n_embd)
self.lm_head = nn.Linear(n_embd, vocab_size)

self.block_size = block_size

def forward(self, idx, targets=None):
    B, T = idx.shape
    assert T <= self.block_size, f"Sequence length {T} exceeds block size {self.block_size}"

    token_emb = self.token_embedding(idx)
    pos = torch.arange(T, device=idx.device).unsqueeze(0) # (1, T)
    pos_emb = self.position_embedding(pos)
    x = token_emb + pos_emb

    x = self.transformer(x)
    x = self.ln_f(x)
    logits = self.lm_head(x)

    loss = None
    if targets is not None:
        B, T, C = logits.shape
        logits = logits.view(B*T, C)
        targets = targets.view(B*T)
        loss = F.cross_entropy(logits, targets)

    return logits, loss

def generate(self, idx, max_new_tokens):
    for _ in range(max_new_tokens):
        idx_cond = idx[:, -self.block_size:]
        logits, _ = self(idx_cond)
        logits = logits[:, -1, :]
        probs = F.softmax(logits, dim=-1)
        idx_next = torch.multinomial(probs, num_samples=1)
        idx = torch.cat((idx, idx_next), dim=1)
    return idx

```

Μπορούμε να παρατηρήσουμε ότι ο κώδικας που έχει δώσει σαν απάντηση για την βελτιστοποίηση του κώδικα μοιάζει αρκετά σε μορφή και λογική με τον επιθυμητό κώδικα για το αντίστοιχο ερώτημα της άσκησης με την χρήση transformer - encoder. Ο κώδικας αυτός είναι σωστός και αρκετά οργανωμένος και μπορεί να αποτελέσει αφετηρία για την διαδικασία βελτιστοποίησης.